physics-ai999x: ONNX formula pack guide

Version 1.3.1. 365 exact formula graphs — every certified closed-form capability of physics-ai999x, compiled from its exact formula (nothing learned, nothing approximate). Float64, standard ONNX operators; run them with onnxruntime in Python, C++, C#, Java or the browser. Creator / engineer: Ibnul Jaif Farabi · AI999X · Apache-2.0.

Find the right formula (autocomplete)

Never guess a formula: search the catalog. `catalog.json` lists every graph with its name, keywords, inputs (meaning and SI unit), output unit and LaTeX. In JavaScript, `formula-pack.js` does the search and runs the graph; `formula-pack.d.ts` gives your editor every id and input name.

```
import { loadPack } from "./formula-pack.js";
const pack = await loadPack("https://huggingface.co/AI999X/physics-
ai999x/resolve/main/onnx/formulas");
pack.search("absolute press");      // autocomplete: best matches first
const r = await pack.solve("absolute_pressure_depth.P", { P0: 2.0, rho: 2.0, g: 2.0, h: 2.0 });
console.log(r.value, r.unit);
```

Python and C#

```
import json, numpy as np, onnxruntime as ort
cat = json.load(open('onnx/formulas/catalog.json'))
e = next(x for x in cat['entries'] if x['id'] == 'absolute_pressure_depth.P')
s = ort.InferenceSession('onnx/formulas/' + e['file'])
print(s.run(None, {n: np.array(2.0) for n in e['inputs']}))
```

```
using Microsoft.ML.OnnxRuntime; using Microsoft.ML.OnnxRuntime.Tensors;
using var s = new InferenceSession("onnx/formulas/absolute_pressure_depth__P.onnx");
var inputs = new[] { NamedOnnxValue.CreateFromTensor("P0", new DenseTensor<double>(new[] { 2.0 },
new int[0])), NamedOnnxValue.CreateFromTensor("rho", new DenseTensor<double>(new[] { 2.0 }, new
int[0])), NamedOnnxValue.CreateFromTensor("g", new DenseTensor<double>(new[] { 2.0 }, new int[0])),
NamedOnnxValue.CreateFromTensor("h", new DenseTensor<double>(new[] { 2.0 }, new int[0])) };
using var r = s.Run(inputs);
```

Rules

- Inputs are numbers in **SI units** (convert first: 72 km/h = 20 m/s).
- A formula with several solutions returns one output per branch; take the branch whose sign the quantity allows (the entry's `sign`; formula-pack.js does it). Two valid branches means the question is ambiguous.
- A formula applies only under its stated conditions (`conditions`): a pendulum formula is for a pendulum.
- Questions in words: use the Python package (`pip install physics-ai999x`), which reads them and refuses what it cannot verify.

Catalogue

Absolute pressure at a depth in a fluid — solve for absolute pressure at the depth — `absolute_pressure_depth.P`

$$P = P_0 + \rho g h$$

input	meaning	unit
`P0`	pressure at the surface	`Pa`
`rho`	density of the fluid	`kg/m^3`
`g`	acceleration	`m/s^2`
`h`	depth below the surface	`m`

- **Output unit:** `Pa`
- **File:** `absolute_pressure_depth__P.onnx`
- **Applies when:** fluid_depth

```
inputs {'P0': 2.0, 'rho': 2.1, 'g': 2.2, 'h': 2.3}
-> 12.626 Pa   (ran on absolute_pressure_depth__P.onnx; matches the Python engine)
```

Absolute pressure at a depth in a fluid — solve for pressure at the surface — `absolute_pressure_depth.P0`

$$P = P_0 + \rho g h$$

input	meaning	unit
`P`	absolute pressure at the depth	`Pa`
`rho`	density of the fluid	`kg/m^3`
`g`	acceleration	`m/s^2`
`h`	depth below the surface	`m`

- **Output unit:** `Pa`
- **File:** `absolute_pressure_depth__P0.onnx`
- **Applies when:** fluid_depth

```
inputs {'P': 0.5, 'rho': 0.6, 'g': 0.7, 'h': 0.8}
-> 0.164 Pa   (ran on absolute_pressure_depth__P0.onnx; matches the Python engine)
```

Absolute pressure at a depth in a fluid — solve for gravity — `absolute_pressure_depth.g`

$$P = P_0 + \rho g h$$

input	meaning	unit
`P`	absolute pressure at the depth	`Pa`

input	meaning	unit
`P0`	pressure at the surface	`Pa`
`rho`	density of the fluid	`kg/m^3`
`h`	depth below the surface	`m`

- **Output unit:** `m/s^2`
- **File:** `absolute_pressure_depth__g.onnx`
- **Applies when:** fluid_depth

Absolute pressure at a depth in a fluid — solve for depth below the surface — `absolute_pressure_depth.h`

$$P = P_0 + \rho g h$$

input	meaning	unit
`P`	absolute pressure at the depth	`Pa`
`P0`	pressure at the surface	`Pa`
`rho`	density of the fluid	`kg/m^3`
`g`	acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `absolute_pressure_depth__h.onnx`
- **Applies when:** fluid_depth

Absolute pressure at a depth in a fluid — solve for density of the fluid — `absolute_pressure_depth.rho`

$$P = P_0 + \rho g h$$

input	meaning	unit
`P`	absolute pressure at the depth	`Pa`
`P0`	pressure at the surface	`Pa`
`g`	acceleration	`m/s^2`
`h`	depth below the surface	`m`

- **Output unit:** `kg/m^3`
- **File:** `absolute_pressure_depth__rho.onnx`
- **Applies when:** fluid_depth

Acceleration down a frictionless incline — solve for acceleration down the incline — `acceleration_frictionless_incline.a`

$$a = g \sin(\theta)$$

input	meaning	unit
`g`	gravitational acceleration	`m/s^2`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `m/s^2`
- **File:** `acceleration_frictionless_incline__a.onnx`
- **Applies when:** inclined; frictionless

```
inputs {'g': 2.0, 'theta': 2.1}
-> 1.72642 m/s^2   (ran on acceleration_frictionless_incline__a.onnx; matches the Python engine)
```

Acceleration down a frictionless incline — solve for gravitational acceleration — `acceleration_frictionless_incline.g`

$$a = g \sin(\theta)$$

input	meaning	unit
`a`	acceleration down the incline	`m/s^2`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `m/s^2`
- **File:** `acceleration_frictionless_incline__g.onnx`
- **Applies when:** inclined; frictionless

```
inputs {'a': 2.0, 'theta': 2.1}
-> 2.31694 m/s^2   (ran on acceleration_frictionless_incline__g.onnx; matches the Python engine)
```

Acceleration down a frictionless incline — solve for angle of the incline to the horizontal — `acceleration_frictionless_incline.theta`

$$a = g \sin(\theta)$$

input	meaning	unit
`a`	acceleration down the incline	`m/s^2`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `rad`
- **File:** `acceleration_frictionless_incline__theta.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** inclined; frictionless

Angular frequency and frequency — solve for frequency — `angular_frequency.f`

$$\omega = 2\pi f$$

input	meaning	unit
`omega`	angular frequency	`rad/s`

- **Output unit:** `Hz`
- **File:** `angular_frequency__f.onnx`

```
inputs {'omega': 2.0}
-> 0.31831 Hz   (ran on angular_frequency__f.onnx; matches the Python engine)
```

Angular frequency and frequency — solve for angular frequency — `angular_frequency.omega`

$$\omega = 2\pi f$$

input	meaning	unit
`f`	frequency	`Hz`

- **Output unit:** `rad/s`
- **File:** `angular_frequency__omega.onnx`

```
inputs {'f': 2.0}
-> 12.5664 rad/s   (ran on angular_frequency__omega.onnx; matches the Python engine)
```

Angular momentum of a rotating body — solve for moment of inertia — `angular_momentum.Ir`

$$L = I_r \omega$$

input	meaning	unit
`L`	angular momentum	`kg*m^2/s`
`omega`	angular velocity	`rad/s`

- **Output unit:** `kg*m^2`
- **File:** `angular_momentum__Ir.onnx`
- **Applies when:** rotating

```
inputs {'L': 2.0, 'omega': 2.1}
-> 0.952381 kg*m^2   (ran on angular_momentum__Ir.onnx; matches the Python engine)
```

Angular momentum of a rotating body — solve for angular momentum — `angular_momentum.L`

$$L = I_r \omega$$

input	meaning	unit
`Ir`	moment of inertia	`kg*m^2`
`omega`	angular velocity	`rad/s`

- **Output unit:** `kg*m^2/s`
- **File:** `angular_momentum__L.onnx`
- **Applies when:** rotating

```
inputs {'Ir': 2.0, 'omega': 2.1}
-> 4.2 kg*m^2/s   (ran on angular_momentum__L.onnx; matches the Python engine)
```

Angular momentum of a rotating body — solve for angular velocity — `angular_momentum.omega`

$$L = I_r \omega$$

input	meaning	unit
`L`	angular momentum	`kg*m^2/s`
`Ir`	moment of inertia	`kg*m^2`

- **Output unit:** `rad/s`
- **File:** `angular_momentum__omega.onnx`
- **Applies when:** rotating

```
inputs {'L': 2.0, 'Ir': 2.1}
-> 0.952381 rad/s   (ran on angular_momentum__omega.onnx; matches the Python engine)
```

Angular momentum of a point mass on a circle — solve for angular momentum about the centre — `angular_momentum_point_mass.L`

$$L = mvr$$

input	meaning	unit
`m`	mass	`kg`
`v`	speed on the circle	`m/s`
`r`	radius of the circle	`m`

- **Output unit:** `kg*m^2/s`
- **File:** `angular_momentum_point_mass__L.onnx`
- **Applies when:** orbit

```
inputs {'m': 2.0, 'v': 2.1, 'r': 2.2}
-> 9.24 kg*m^2/s   (ran on angular_momentum_point_mass__L.onnx; matches the Python engine)
```

Angular momentum of a point mass on a circle — solve for mass — `angular_momentum_point_mass.m`

$$L = mvr$$

input	meaning	unit
`L`	angular momentum about the centre	`kg*m^2/s`
`v`	speed on the circle	`m/s`
`r`	radius of the circle	`m`

- **Output unit:** `kg`
- **File:** `angular_momentum_point_mass__m.onnx`
- **Applies when:** orbit

```
inputs {'L': 2.0, 'v': 2.1, 'r': 2.2}
-> 0.4329 kg   (ran on angular_momentum_point_mass__m.onnx; matches the Python engine)
```

Angular momentum of a point mass on a circle — solve for radius of the circle — `angular_momentum_point_mass.r`

$$L = mvr$$

input	meaning	unit
`L`	angular momentum about the centre	`kg*m^2/s`
`m`	mass	`kg`
`v`	speed on the circle	`m/s`

- **Output unit:** `m`
- **File:** `angular_momentum_point_mass__r.onnx`
- **Applies when:** orbit

```
inputs {'L': 2.0, 'm': 2.1, 'v': 2.2}
-> 0.4329 m   (ran on angular_momentum_point_mass__r.onnx; matches the Python engine)
```

Angular momentum of a point mass on a circle — solve for speed on the circle — `angular_momentum_point_mass.v`

$$L = mvr$$

input	meaning	unit
`L`	angular momentum about the centre	`kg*m^2/s`
`m`	mass	`kg`
`r`	radius of the circle	`m`

- **Output unit:** `m/s`
- **File:** `angular_momentum_point_mass__v.onnx`
- **Applies when:** orbit

```
inputs {'L': 2.0, 'm': 2.1, 'r': 2.2}
-> 0.4329 m/s   (ran on angular_momentum_point_mass__v.onnx; matches the Python engine)
```

Angular velocity and angle turned under constant angular acceleration — solve for constant angular acceleration — `angular_velocity_angle.alpha`

$$\omega^2 = \omega_0^2 + 2\alpha\theta$$

input	meaning	unit
`omega`	final angular speed	`rad/s`
`omega0`	initial angular speed	`rad/s`
`theta`	angle turned	`rad`

- **Output unit:** `rad/s^2`
- **File:** `angular_velocity_angle__alpha.onnx`
- **Applies when:** rotating

```
inputs {'omega': 2.0, 'omega0': 2.1, 'theta': 2.2}
-> -0.0931818 rad/s^2   (ran on angular_velocity_angle__alpha.onnx; matches the Python engine)
```

Angular velocity and angle turned under constant angular acceleration — solve for final angular speed — `angular_velocity_angle.omega`

$$\omega^2 = \omega_0^2 + 2\alpha\theta$$

input	meaning	unit
`omega0`	initial angular speed	`rad/s`
`alpha`	constant angular acceleration	`rad/s^2`
`theta`	angle turned	`rad`

- **Output unit:** `rad/s`
- **File:** `angular_velocity_angle__omega.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** rotating

```
inputs {'omega0': 2.0, 'alpha': 2.1, 'theta': 2.2}
-> -3.63868, 3.63868 rad/s   (ran on angular_velocity_angle__omega.onnx; matches the Python engine)
```

Angular velocity and angle turned under constant angular acceleration — solve for initial angular speed — `angular_velocity_angle.omega0`

$$\omega^2 = \omega_0^2 + 2\alpha\theta$$

input	meaning	unit
`omega`	final angular speed	`rad/s`
`alpha`	constant angular acceleration	`rad/s^2`
`theta`	angle turned	`rad`

- **Output unit:** `rad/s`
- **File:** `angular_velocity_angle__omega0.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** rotating

Angular velocity and angle turned under constant angular acceleration — solve for angle turned — `angular_velocity_angle.theta`

$$\omega^2 = \omega_0^2 + 2\alpha\theta$$

input	meaning	unit
`omega`	final angular speed	`rad/s`
`omega0`	initial angular speed	`rad/s`
`alpha`	constant angular acceleration	`rad/s^2`

- **Output unit:** `rad`
- **File:** `angular_velocity_angle__theta.onnx`
- **Applies when:** rotating

```
inputs {'omega': 2.0, 'omega0': 2.1, 'alpha': 2.2}
-> -0.0931818 rad   (ran on angular_velocity_angle__theta.onnx; matches the Python engine)
```

Angular velocity under constant angular acceleration — solve for constant angular acceleration — `angular_velocity_time.alpha`

$$\omega = \omega_0 + \alpha t$$

input	meaning	unit
`omega`	final angular velocity	`rad/s`
`omega0`	initial angular velocity	`rad/s`
`t`	elapsed time	`s`

- **Output unit:** `rad/s^2`
- **File:** `angular_velocity_time__alpha.onnx`
- **Applies when:** rotating

```
inputs {'omega': 2.0, 'omega0': 2.1, 't': 2.2}
-> -0.0454545 rad/s^2   (ran on angular_velocity_time__alpha.onnx; matches the Python engine)
```

Angular velocity under constant angular acceleration — solve for final angular velocity — `angular_velocity_time.omega`

$$\omega = \omega_0 + \alpha t$$

input	meaning	unit
`omega0`	initial angular velocity	`rad/s`
`alpha`	constant angular acceleration	`rad/s^2`
`t`	elapsed time	`s`

- **Output unit:** `rad/s`
- **File:** `angular_velocity_time__omega.onnx`
- **Applies when:** rotating

```
inputs {'omega0': 2.0, 'alpha': 2.1, 't': 2.2}
-> 6.62 rad/s   (ran on angular_velocity_time__omega.onnx; matches the Python engine)
```

Angular velocity under constant angular acceleration — solve for initial angular velocity — `angular_velocity_time.omega0`

$$\omega = \omega_0 + \alpha t$$

input	meaning	unit
`omega`	final angular velocity	`rad/s`
`alpha`	constant angular acceleration	`rad/s^2`
`t`	elapsed time	`s`

- **Output unit:** `rad/s`
- **File:** `angular_velocity_time__omega0.onnx`
- **Applies when:** rotating

```
inputs {'omega': 2.0, 'alpha': 2.1, 't': 2.2}
-> -2.62 rad/s   (ran on angular_velocity_time__omega0.onnx; matches the Python engine)
```

Angular velocity under constant angular acceleration — solve for elapsed time — `angular_velocity_time.t`

$$\omega = \omega_0 + \alpha t$$

input	meaning	unit
`omega`	final angular velocity	`rad/s`
`omega0`	initial angular velocity	`rad/s`
`alpha`	constant angular acceleration	`rad/s^2`

- **Output unit:** `s`
- **File:** `angular_velocity_time__t.onnx`
- **Applies when:** rotating

Thermal expansion of an area — solve for original area — `area_expansion.A`

$$d_A = 2\alpha_t A d_T$$

input	meaning	unit
`dA`	change in area	`m^2`
`alpha_t`	coefficient of linear expansion	`1/K`
`dT`	change in temperature	`K`

- **Output unit:** `m^2`
- **File:** `area_expansion__A.onnx`
- **Applies when:** area_expansion

```
inputs {'dA': 2.0, 'alpha_t': 2.1, 'dT': 2.2}
-> 0.21645 m^2   (ran on area_expansion__A.onnx; matches the Python engine)
```

Thermal expansion of an area — solve for coefficient of linear expansion — `area_expansion.alpha_t`

$$d_A = 2\alpha_t A d_T$$

input	meaning	unit
`dA`	change in area	`m^2`
`A`	original area	`m^2`
`dT`	change in temperature	`K`

- **Output unit:** `1/K`
- **File:** `area_expansion__alpha_t.onnx`
- **Applies when:** area_expansion

```
inputs {'dA': 2.0, 'A': 2.1, 'dT': 2.2}
-> 0.21645 1/K   (ran on area_expansion__alpha_t.onnx; matches the Python engine)
```

Thermal expansion of an area — solve for change in area — `area_expansion.dA`

$$d_A = 2\alpha_t A d_T$$

input	meaning	unit
`alpha_t`	coefficient of linear expansion	`1/K`
`A`	original area	`m^2`
`dT`	change in temperature	`K`

- **Output unit:** `m^2`
- **File:** `area_expansion__dA.onnx`

- **Applies when:** area_expansion

```
inputs {'alpha_t': 2.0, 'A': 2.1, 'dT': 2.2}
-> 18.48 m^2   (ran on area_expansion__dA.onnx; matches the Python engine)
```

Thermal expansion of an area — solve for change in temperature — `area_expansion.dT`

$$d_A = 2\alpha_t A d_T$$

input	meaning	unit
`dA`	change in area	`m^2`
`alpha_t`	coefficient of linear expansion	`1/K`
`A`	original area	`m^2`

- **Output unit:** `K`
- **File:** `area_expansion__dT.onnx`
- **Applies when:** area_expansion

```
inputs {'dA': 2.0, 'alpha_t': 2.1, 'A': 2.2}
-> 0.21645 K   (ran on area_expansion__dT.onnx; matches the Python engine)
```

Average net force from a change of momentum — solve for average net force — `average_force_from_momentum_change.F`

$$F = \frac{p_2 - p_1}{t}$$

input	meaning	unit
`p2`	final momentum	`kg*m/s`
`p1`	initial momentum	`kg*m/s`
`t`	time over which the momentum changes	`s`

- **Output unit:** `N`
- **File:** `average_force_from_momentum_change__F.onnx`

```
inputs {'p2': 2.0, 'p1': 2.1, 't': 2.2}
-> -0.0454545 N   (ran on average_force_from_momentum_change__F.onnx; matches the Python engine)
```

Average net force from a change of momentum — solve for initial momentum — `average_force_from_momentum_change.p1`

$$F = \frac{p_2 - p_1}{t}$$

input	meaning	unit
`F`	average net force	`N`
`p2`	final momentum	`kg*m/s`
`t`	time over which the momentum changes	`s`

- **Output unit:** `kg*m/s`
- **File:** `average_force_from_momentum_change__p1.onnx`

```
inputs {'F': 2.0, 'p2': 2.1, 't': 2.2}
-> -2.3 kg*m/s   (ran on average_force_from_momentum_change__p1.onnx; matches the Python engine)
```

Average net force from a change of momentum — solve for final momentum — `average_force_from_momentum_change.p2`

$$F = \frac{p_2 - p_1}{t}$$

input	meaning	unit
`F`	average net force	`N`
`p1`	initial momentum	`kg*m/s`
`t`	time over which the momentum changes	`s`

- **Output unit:** `kg*m/s`
- **File:** `average_force_from_momentum_change__p2.onnx`

```
inputs {'F': 2.0, 'p1': 2.1, 't': 2.2}
-> 6.5 kg*m/s   (ran on average_force_from_momentum_change__p2.onnx; matches the Python engine)
```

Average net force from a change of momentum — solve for time over which the momentum changes — `average_force_from_momentum_change.t`

$$F = \frac{p_2 - p_1}{t}$$

input	meaning	unit
`F`	average net force	`N`
`p2`	final momentum	`kg*m/s`
`p1`	initial momentum	`kg*m/s`

- **Output unit:** `s`
- **File:** `average_force_from_momentum_change__t.onnx`

Displacement from the mean of initial and final velocity — solve for elapsed time — `average_velocity_displacement.t`

$$x = \frac{(v_0 + v)t}{2}$$

input	meaning	unit
`x`	displacement	`m`
`v0`	initial velocity	`m/s`
`v`	final velocity	`m/s`

- **Output unit:** `s`
- **File:** `average_velocity_displacement__t.onnx`

```
inputs {'x': 2.0, 'v0': 2.1, 'v': 2.2}
-> 0.930233 s   (ran on average_velocity_displacement__t.onnx; matches the Python engine)
```

Displacement from the mean of initial and final velocity — solve for final velocity — `average_velocity_displacement.v`

$$x = \frac{(v_0 + v)t}{2}$$

input	meaning	unit
`x`	displacement	`m`
`v0`	initial velocity	`m/s`
`t`	elapsed time	`s`

- **Output unit:** `m/s`
- **File:** `average_velocity_displacement__v.onnx`

```
inputs {'x': 2.0, 'v0': 2.1, 't': 2.2}
-> -0.281818 m/s   (ran on average_velocity_displacement__v.onnx; matches the Python engine)
```

Displacement from the mean of initial and final velocity — solve for initial velocity — `average_velocity_displacement.v0`

$$x = \frac{(v_0 + v)t}{2}$$

input	meaning	unit
`x`	displacement	`m`
`v`	final velocity	`m/s`
`t`	elapsed time	`s`

- **Output unit:** `m/s`
- **File:** `average_velocity_displacement__v0.onnx`

```
inputs {'x': 2.0, 'v': 2.1, 't': 2.2}
-> -0.281818 m/s   (ran on average_velocity_displacement__v0.onnx; matches the Python engine)
```

Displacement from the mean of initial and final velocity — solve for displacement — `average_velocity_displacement.x`

$$x = \frac{(v_0 + v)t}{2}$$

input	meaning	unit
`v0`	initial velocity	`m/s`
`v`	final velocity	`m/s`
`t`	elapsed time	`s`

- **Output unit:** `m`
- **File:** `average_velocity_displacement__x.onnx`

```
inputs {'v0': 2.0, 'v': 2.1, 't': 2.2}
-> 4.51 m   (ran on average_velocity_displacement__x.onnx; matches the Python engine)
```

Charge on a capacitor — solve for capacitance — `capacitance_definition.C`

$$Q = CV$$

input	meaning	unit
`Q`	charge on either plate	`C`
`V`	voltage across the capacitor	`V`

- **Output unit:** `F`
- **File:** `capacitance_definition__C.onnx`
- **Applies when:** capacitor

```
inputs {'Q': 2.0, 'V': 2.1}
-> 0.952381 F   (ran on capacitance_definition__C.onnx; matches the Python engine)
```

Charge on a capacitor — solve for charge on either plate — `capacitance_definition.Q`

$$Q = CV$$

input	meaning	unit
`C`	capacitance	`F`
`V`	voltage across the capacitor	`V`

- **Output unit:** `C`
- **File:** `capacitance_definition__Q.onnx`
- **Applies when:** capacitor

```
inputs {'C': 2.0, 'V': 2.1}
-> 4.2 C   (ran on capacitance_definition__Q.onnx; matches the Python engine)
```

Charge on a capacitor — solve for voltage across the capacitor — `capacitance_definition.V`

$$Q = CV$$

input	meaning	unit
`Q`	charge on either plate	`C`
`C`	capacitance	`F`

- **Output unit:** `V`
- **File:** `capacitance_definition__V.onnx`
- **Applies when:** capacitor

```
inputs {'Q': 2.0, 'C': 2.1}
-> 0.952381 V   (ran on capacitance_definition__V.onnx; matches the Python engine)
```

Reactance of a capacitor — solve for capacitance — `capacitive_reactance.C`

$$X = \frac{1}{2\pi f C}$$

input	meaning	unit
`X`	capacitive reactance	`ohm`
`f`	frequency	`Hz`

- **Output unit:** `F`
- **File:** `capacitive_reactance__C.onnx`
- **Applies when:** capacitor

```
inputs {'X': 2.0, 'f': 2.1}
-> 0.037894 F   (ran on capacitive_reactance__C.onnx; matches the Python engine)
```

Reactance of a capacitor — solve for capacitive reactance — `capacitive_reactance.X`

$$X = \frac{1}{2\pi f C}$$

input	meaning	unit
`f`	frequency	`Hz`

input	meaning	unit
`C`	capacitance	`F`

- **Output unit:** `ohm`
- **File:** `capacitive_reactance__X.onnx`
- **Applies when:** capacitor

```
inputs {'f': 2.0, 'C': 2.1}
-> 0.037894 ohm   (ran on capacitive_reactance__X.onnx; matches the Python engine)
```

Reactance of a capacitor — solve for frequency — `capacitive_reactance.f`

$$X = \frac{1}{2\pi f C}$$

input	meaning	unit
`X`	capacitive reactance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `Hz`
- **File:** `capacitive_reactance__f.onnx`
- **Applies when:** capacitor

```
inputs {'X': 2.0, 'C': 2.1}
-> 0.037894 Hz   (ran on capacitive_reactance__f.onnx; matches the Python engine)
```

Energy stored in a capacitor — solve for capacitance — `capacitor_energy.C`

$$\mathrm{Ecap} = \frac{CV^2}{2}$$

input	meaning	unit
`Ecap`	energy stored	`J`
`V`	voltage	`V`

- **Output unit:** `F`
- **File:** `capacitor_energy__C.onnx`
- **Applies when:** capacitor

```
inputs {'Ecap': 2.0, 'V': 2.1}
-> 0.907029 F   (ran on capacitor_energy__C.onnx; matches the Python engine)
```

Energy stored in a capacitor — solve for energy stored — `capacitor_energy.Ecap`

$$\mathrm{Ecap} = \frac{CV^2}{2}$$

input	meaning	unit
`C`	capacitance	`F`
`V`	voltage	`V`

- **Output unit:** `J`
- **File:** `capacitor_energy__Ecap.onnx`
- **Applies when:** capacitor

```
inputs {'C': 2.0, 'V': 2.1}
-> 4.41 J   (ran on capacitor_energy__Ecap.onnx; matches the Python engine)
```

Energy stored in a capacitor — solve for voltage — `capacitor_energy.V`

$$\mathrm{Ecap} = \frac{CV^2}{2}$$

input	meaning	unit
`Ecap`	energy stored	`J`
`C`	capacitance	`F`

- **Output unit:** `V`
- **File:** `capacitor_energy__V.onnx`
- **Applies when:** capacitor

```
inputs {'Ecap': 2.0, 'C': 2.1}
-> 1.38013 V   (ran on capacitor_energy__V.onnx; matches the Python engine)
```

Carnot (maximum) efficiency — solve for cold reservoir temperature (absolute) — `carnot_efficiency.Tc`

$$\mathrm{eff} = 1 - \frac{T_c}{T_h}$$

input	meaning	unit
`eff`	maximum efficiency	`1`
`Th`	hot reservoir temperature (absolute)	`K`

- **Output unit:** `K`
- **File:** `carnot_efficiency__Tc.onnx`
- **Applies when:** heat_engine

```
inputs {'eff': 0.5, 'Th': 0.6}
-> 0.3 K   (ran on carnot_efficiency__Tc.onnx; matches the Python engine)
```

Carnot (maximum) efficiency — solve for hot reservoir temperature (absolute) — `carnot_efficiency.Th`

$$\mathrm{eff} = 1 - \frac{T_c}{T_h}$$

input	meaning	unit
`eff`	maximum efficiency	`1`
`Tc`	cold reservoir temperature (absolute)	`K`

- **Output unit:** `K`
- **File:** `carnot_efficiency__Th.onnx`
- **Applies when:** heat_engine

```
inputs {'eff': 0.5, 'Tc': 0.6}
-> 1.2 K   (ran on carnot_efficiency__Th.onnx; matches the Python engine)
```

Carnot (maximum) efficiency — solve for maximum efficiency — `carnot_efficiency.eff`

$$\mathrm{eff} = 1 - \frac{T_c}{T_h}$$

input	meaning	unit
`Tc`	cold reservoir temperature (absolute)	`K`
`Th`	hot reservoir temperature (absolute)	`K`

- **Output unit:** `1`
- **File:** `carnot_efficiency__eff.onnx`
- **Applies when:** heat_engine

```
inputs {'Tc': 2.0, 'Th': 2.1}
-> 0.047619 1   (ran on carnot_efficiency__eff.onnx; matches the Python engine)
```

Centripetal acceleration — solve for centripetal acceleration — `centripetal_acceleration.a`

$$a = \frac{v^2}{r}$$

input	meaning	unit
`v`	speed	`m/s`
`r`	radius of the circle	`m`

- **Output unit:** `m/s^2`
- **File:** `centripetal_acceleration__a.onnx`

```
inputs {'v': 2.0, 'r': 2.1}
-> 1.90476 m/s^2   (ran on centripetal_acceleration__a.onnx; matches the Python engine)
```

Centripetal acceleration — solve for radius of the circle — `centripetal_acceleration.r`

$$a = \frac{v^2}{r}$$

input	meaning	unit
`a`	centripetal acceleration	`m/s^2`
`v`	speed	`m/s`

- **Output unit:** `m`
- **File:** `centripetal_acceleration__r.onnx`

```
inputs {'a': 2.0, 'v': 2.1}
-> 2.205 m   (ran on centripetal_acceleration__r.onnx; matches the Python engine)
```

Centripetal acceleration — solve for speed — `centripetal_acceleration.v`

$$a = \frac{v^2}{r}$$

input	meaning	unit
`a`	centripetal acceleration	`m/s^2`
`r`	radius of the circle	`m`

- **Output unit:** `m/s`
- **File:** `centripetal_acceleration__v.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'a': 2.0, 'r': 2.1}
-> -2.04939, 2.04939 m/s   (ran on centripetal_acceleration__v.onnx; matches the Python engine)
```

Centripetal acceleration from angular velocity — solve for centripetal acceleration — `centripetal_acceleration_omega.a`

$$a = r\omega^2$$

input	meaning	unit
`r`	radius of the circle	`m`
`omega`	angular velocity	`rad/s`

- **Output unit:** `m/s^2`
- **File:** `centripetal_acceleration_omega__a.onnx`

- **Applies when:** rotating

```
inputs {'r': 2.0, 'omega': 2.1}
-> 8.82 m/s^2   (ran on centripetal_acceleration_omega__a.onnx; matches the Python engine)
```

Centripetal acceleration from angular velocity — solve for angular velocity — `centripetal_acceleration_omega.omega`

$$a = r\omega^2$$

input	meaning	unit
`a`	centripetal acceleration	`m/s^2`
`r`	radius of the circle	`m`

- **Output unit:** `rad/s`
- **File:** `centripetal_acceleration_omega__omega.onnx`
- **Applies when:** rotating

```
inputs {'a': 2.0, 'r': 2.1}
-> 0.9759 rad/s   (ran on centripetal_acceleration_omega__omega.onnx; matches the Python engine)
```

Centripetal acceleration from angular velocity — solve for radius of the circle — `centripetal_acceleration_omega.r`

$$a = r\omega^2$$

input	meaning	unit
`a`	centripetal acceleration	`m/s^2`
`omega`	angular velocity	`rad/s`

- **Output unit:** `m`
- **File:** `centripetal_acceleration_omega__r.onnx`
- **Applies when:** rotating

```
inputs {'a': 2.0, 'omega': 2.1}
-> 0.453515 m   (ran on centripetal_acceleration_omega__r.onnx; matches the Python engine)
```

Centripetal force — solve for centripetal force — `centripetal_force.F`

$$F = \frac{mv^2}{r}$$

input	meaning	unit
`m`	mass	`kg`
`v`	speed	`m/s`

input	meaning	unit
`r`	radius of the circle	`m`

- **Output unit:** `N`
- **File:** `centripetal_force__F.onnx`

```
inputs {'m': 2.0, 'v': 2.1, 'r': 2.2}
-> 4.00909 N   (ran on centripetal_force__F.onnx; matches the Python engine)
```

Centripetal force — solve for mass — `centripetal_force.m`

$$F = \frac{mv^2}{r}$$

input	meaning	unit
`F`	centripetal force	`N`
`v`	speed	`m/s`
`r`	radius of the circle	`m`

- **Output unit:** `kg`
- **File:** `centripetal_force__m.onnx`

```
inputs {'F': 2.0, 'v': 2.1, 'r': 2.2}
-> 0.997732 kg   (ran on centripetal_force__m.onnx; matches the Python engine)
```

Centripetal force — solve for radius of the circle — `centripetal_force.r`

$$F = \frac{mv^2}{r}$$

input	meaning	unit
`F`	centripetal force	`N`
`m`	mass	`kg`
`v`	speed	`m/s`

- **Output unit:** `m`
- **File:** `centripetal_force__r.onnx`

```
inputs {'F': 2.0, 'm': 2.1, 'v': 2.2}
-> 5.082 m   (ran on centripetal_force__r.onnx; matches the Python engine)
```

Centripetal force — solve for speed — `centripetal_force.v`

$$F = \frac{mv^2}{r}$$

input	meaning	unit
`F`	centripetal force	`N`
`m`	mass	`kg`
`r`	radius of the circle	`m`

- **Output unit:** `m/s`
- **File:** `centripetal_force__v.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'F': 2.0, 'm': 2.1, 'r': 2.2}
-> -1.44749, 1.44749 m/s   (ran on centripetal_force__v.onnx; matches the Python engine)
```

Charge carried by a steady current — solve for current — `charge_current.I`

$$Q = It$$

input	meaning	unit
`Q`	charge	`C`
`t`	elapsed time	`s`

- **Output unit:** `A`
- **File:** `charge_current__I.onnx`

```
inputs {'Q': 2.0, 't': 2.1}
-> 0.952381 A   (ran on charge_current__I.onnx; matches the Python engine)
```

Charge carried by a steady current — solve for charge — `charge_current.Q`

$$Q = It$$

input	meaning	unit
`I`	current	`A`
`t`	elapsed time	`s`

- **Output unit:** `C`
- **File:** `charge_current__Q.onnx`

```
inputs {'I': 2.0, 't': 2.1}
-> 4.2 C   (ran on charge_current__Q.onnx; matches the Python engine)
```

Charge carried by a steady current — solve for elapsed time — `charge_current.t`

$$Q = It$$

input	meaning	unit
`Q`	charge	`C`
`I`	current	`A`

- **Output unit:** `s`
- **File:** `charge_current__t.onnx`

```
inputs {'Q': 2.0, 'I': 2.1}
-> 0.952381 s   (ran on charge_current__t.onnx; matches the Python engine)
```

Coefficient of performance of a heat pump — solve for heat delivered to the warm space — `cop_heat_pump.Qh`

$$\mathrm{cop} = \frac{Q_h}{W}$$

input	meaning	unit
`cop`	coefficient of performance	`1`
`W`	work input	`J`

- **Output unit:** `J`
- **File:** `cop_heat_pump__Qh.onnx`
- **Applies when:** heat_pump

```
inputs {'cop': 2.0, 'W': 2.1}
-> 4.2 J   (ran on cop_heat_pump__Qh.onnx; matches the Python engine)
```

Coefficient of performance of a heat pump — solve for work input — `cop_heat_pump.W`

$$\mathrm{cop} = \frac{Q_h}{W}$$

input	meaning	unit
`cop`	coefficient of performance	`1`
`Qh`	heat delivered to the warm space	`J`

- **Output unit:** `J`
- **File:** `cop_heat_pump__W.onnx`
- **Applies when:** heat_pump

```
inputs {'cop': 2.0, 'Qh': 2.1}
-> 1.05 J   (ran on cop_heat_pump__W.onnx; matches the Python engine)
```

Coefficient of performance of a heat pump — solve for coefficient of performance — `cop_heat_pump.cop`

$$\text{cop} = \frac{Q_h}{W}$$

input	meaning	unit
`Qh`	heat delivered to the warm space	`J`
`W`	work input	`J`

- **Output unit:** `1`
- **File:** `cop_heat_pump__cop.onnx`
- **Applies when:** heat_pump

```
inputs {'Qh': 2.0, 'W': 2.1}
-> 0.952381 1   (ran on cop_heat_pump__cop.onnx; matches the Python engine)
```

Coefficient of performance of a refrigerator — solve for heat removed from the cold space — `cop_refrigerator.Qc`

$$\text{cop} = \frac{Q_c}{W}$$

input	meaning	unit
`cop`	coefficient of performance	`1`
`W`	work input	`J`

- **Output unit:** `J`
- **File:** `cop_refrigerator__Qc.onnx`
- **Applies when:** refrigerator

```
inputs {'cop': 2.0, 'W': 2.1}
-> 4.2 J   (ran on cop_refrigerator__Qc.onnx; matches the Python engine)
```

Coefficient of performance of a refrigerator — solve for work input — `cop_refrigerator.W`

$$\text{cop} = \frac{Q_c}{W}$$

input	meaning	unit
`cop`	coefficient of performance	`1`
`Qc`	heat removed from the cold space	`J`

- **Output unit:** `J`
- **File:** `cop_refrigerator__W.onnx`
- **Applies when:** refrigerator

```
inputs {'cop': 2.0, 'Qc': 2.1}
-> 1.05 J   (ran on cop_refrigerator__W.onnx; matches the Python engine)
```

Coefficient of performance of a refrigerator — solve for coefficient of performance — `cop_refrigerator.cop`

$$\text{cop} = \frac{Q_c}{W}$$

input	meaning	unit
`Qc`	heat removed from the cold space	`J`
`W`	work input	`J`

- **Output unit:** `1`
- **File:** `cop_refrigerator__cop.onnx`
- **Applies when:** refrigerator

```
inputs {'Qc': 2.0, 'W': 2.1}
-> 0.952381 1   (ran on cop_refrigerator__cop.onnx; matches the Python engine)
```

Radius of a charged particle's circle in a magnetic field — solve for field strength — `cyclotron_radius.B`

$$r = \frac{mv}{qB\sin(\theta)}$$

input	meaning	unit
`r`	radius of the circular path	`m`
`m`	mass of the particle	`kg`
`v`	speed perpendicular to the field	`m/s`
`q`	magnitude of the charge	`C`
`theta`	angle between the velocity and the field	`rad`

- **Output unit:** `T`
- **File:** `cyclotron_radius__B.onnx`
- **Applies when:** magnetic; charged_particle

```
inputs {'r': 2.0, 'm': 2.1, 'v': 2.2, 'q': 2.3, 'theta': 2.4}
-> 1.4869 T   (ran on cyclotron_radius__B.onnx; matches the Python engine)
```

Radius of a charged particle's circle in a magnetic field — solve for mass of the particle — `cyclotron_radius.m`

$$r = \frac{mv}{qB\sin(\theta)}$$

input	meaning	unit
`r`	radius of the circular path	`m`
`v`	speed perpendicular to the field	`m/s`
`q`	magnitude of the charge	`C`
`B`	field strength	`T`
`theta`	angle between the velocity and the field	`rad`

- **Output unit:** `kg`
- **File:** `cyclotron_radius__m.onnx`
- **Applies when:** magnetic; charged_particle

```
inputs {'r': 2.0, 'v': 2.1, 'q': 2.2, 'B': 2.3, 'theta': 2.4}
-> 3.25509 kg   (ran on cyclotron_radius__m.onnx; matches the Python engine)
```

Radius of a charged particle's circle in a magnetic field — solve for magnitude of the charge — `cyclotron_radius.q`

$$r = \frac{mv}{qB\sin\left(\theta\right)}$$

input	meaning	unit
`r`	radius of the circular path	`m`
`m`	mass of the particle	`kg`
`v`	speed perpendicular to the field	`m/s`
`B`	field strength	`T`
`theta`	angle between the velocity and the field	`rad`

- **Output unit:** `C`
- **File:** `cyclotron_radius__q.onnx`
- **Applies when:** magnetic; charged_particle

```
inputs {'r': 2.0, 'm': 2.1, 'v': 2.2, 'B': 2.3, 'theta': 2.4}
-> 1.4869 C   (ran on cyclotron_radius__q.onnx; matches the Python engine)
```

Radius of a charged particle's circle in a magnetic field — solve for radius of the circular path — `cyclotron_radius.r`

$$r = \frac{mv}{qB\sin\left(\theta\right)}$$

input	meaning	unit
`m`	mass of the particle	`kg`
`v`	speed perpendicular to the field	`m/s`
`q`	magnitude of the charge	`C`

input	meaning	unit
`B`	field strength	`T`
`theta`	angle between the velocity and the field	`rad`

- **Output unit:** `m`
- **File:** `cyclotron_radius__r.onnx`
- **Applies when:** magnetic; charged_particle

```
inputs {'m': 2.0, 'v': 2.1, 'q': 2.2, 'B': 2.3, 'theta': 2.4}
-> 1.22884 m   (ran on cyclotron_radius__r.onnx; matches the Python engine)
```

Radius of a charged particle's circle in a magnetic field — solve for angle between the velocity and the field — `cyclotron_radius.theta`

$$r = \frac{mv}{qB\sin(\theta)}$$

input	meaning	unit
`r`	radius of the circular path	`m`
`m`	mass of the particle	`kg`
`v`	speed perpendicular to the field	`m/s`
`q`	magnitude of the charge	`C`
`B`	field strength	`T`

- **Output unit:** `rad`
- **File:** `cyclotron_radius__theta.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** magnetic; charged_particle

Radius of a charged particle's circle in a magnetic field — solve for speed perpendicular to the field — `cyclotron_radius.v`

$$r = \frac{mv}{qB\sin(\theta)}$$

input	meaning	unit
`r`	radius of the circular path	`m`
`m`	mass of the particle	`kg`
`q`	magnitude of the charge	`C`
`B`	field strength	`T`
`theta`	angle between the velocity and the field	`rad`

- **Output unit:** `m/s`
- **File:** `cyclotron_radius__v.onnx`
- **Applies when:** magnetic; charged_particle

```
inputs {'r': 2.0, 'm': 2.1, 'q': 2.2, 'B': 2.3, 'theta': 2.4}
-> 3.25509 m/s   (ran on cyclotron_radius__v.onnx; matches the Python engine)
```

De Broglie wavelength — solve for constant — `de_broglie.h`

$$\lambda = \frac{h}{p}$$

input	meaning	unit
`lam`	de Broglie wavelength	`m`
`p`	momentum of the particle	`kg*m/s`

- **Output unit:** `J*s`
- **File:** `de_broglie__h.onnx`
- **Applies when:** matter_wave

```
inputs {'lam': 2.0, 'p': 2.1}
-> 4.2 J*s   (ran on de_broglie__h.onnx; matches the Python engine)
```

De Broglie wavelength — solve for de Broglie wavelength — `de_broglie.lam`

$$\lambda = \frac{h}{p}$$

input	meaning	unit
`p`	momentum of the particle	`kg*m/s`
`h`	planck constant	`J*s`

- **Output unit:** `m`
- **File:** `de_broglie__lam.onnx`
- **Applies when:** matter_wave

```
inputs {'p': 2.0, 'h': 2.1}
-> 1.05 m   (ran on de_broglie__lam.onnx; matches the Python engine)
```

De Broglie wavelength — solve for momentum of the particle — `de_broglie.p`

$$\lambda = \frac{h}{p}$$

input	meaning	unit
`lam`	de Broglie wavelength	`m`
`h`	planck constant	`J*s`

- **Output unit:** `kg*m/s`
- **File:** `de_broglie__p.onnx`

- **Applies when:** matter_wave

```
inputs {'lam': 2.0, 'h': 2.1}
-> 1.05 kg*m/s   (ran on de_broglie__p.onnx; matches the Python engine)
```

Mass density — solve for volume — `density.V`

$$\rho = \frac{m}{V}$$

input	meaning	unit
`rho`	density	`kg/m^3`
`m`	mass	`kg`

- **Output unit:** `m^3`
- **File:** `density__V.onnx`

```
inputs {'rho': 2.0, 'm': 2.1}
-> 1.05 m^3   (ran on density__V.onnx; matches the Python engine)
```

Mass density — solve for mass — `density.m`

$$\rho = \frac{m}{V}$$

input	meaning	unit
`rho`	density	`kg/m^3`
`V`	volume	`m^3`

- **Output unit:** `kg`
- **File:** `density__m.onnx`

```
inputs {'rho': 2.0, 'V': 2.1}
-> 4.2 kg   (ran on density__m.onnx; matches the Python engine)
```

Mass density — solve for density — `density.rho`

$$\rho = \frac{m}{V}$$

input	meaning	unit
`m`	mass	`kg`
`V`	volume	`m^3`

- **Output unit:** `kg/m^3`
- **File:** `density__rho.onnx`

```
inputs {'m': 2.0, 'V': 2.1}
-> 0.952381 kg/m^3   (ran on density__rho.onnx; matches the Python engine)
```

Displacement under constant acceleration — solve for acceleration — `displacement_time.a`

$$x = v_0 t + \frac{a t^2}{2}$$

input	meaning	unit
`x`	displacement	`m`
`v0`	initial velocity	`m/s`
`t`	elapsed time	`s`

- **Output unit:** `m/s^2`
- **File:** `displacement_time__a.onnx`

```
inputs {'x': 2.0, 'v0': 2.1, 't': 2.2}
-> -1.08264 m/s^2   (ran on displacement_time__a.onnx; matches the Python engine)
```

Displacement under constant acceleration — solve for elapsed time — `displacement_time.t`

$$x = v_0 t + \frac{a t^2}{2}$$

input	meaning	unit
`x`	displacement	`m`
`v0`	initial velocity	`m/s`
`a`	acceleration	`m/s^2`

- **Output unit:** `s`
- **File:** `displacement_time__t.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'x': 2.0, 'v0': 2.1, 'a': 2.2}
-> -2.60662, 0.697526 s   (ran on displacement_time__t.onnx; matches the Python engine)
```

Displacement under constant acceleration — solve for initial velocity — `displacement_time.v0`

$$x = v_0 t + \frac{a t^2}{2}$$

input	meaning	unit
`x`	displacement	`m`
`a`	acceleration	`m/s^2`
`t`	elapsed time	`s`

- **Output unit:** `m/s`
- **File:** `displacement_time__v0.onnx`

```
inputs {'x': 2.0, 'a': 2.1, 't': 2.2}
-> -1.40091 m/s   (ran on displacement_time__v0.onnx; matches the Python engine)
```

Displacement under constant acceleration — solve for displacement — `displacement_time.x`

$$x = v_0 t + \frac{a t^2}{2}$$

input	meaning	unit
`v0`	initial velocity	`m/s`
`a`	acceleration	`m/s^2`
`t`	elapsed time	`s`

- **Output unit:** `m`
- **File:** `displacement_time__x.onnx`

```
inputs {'v0': 2.0, 'a': 2.1, 't': 2.2}
-> 9.482 m   (ran on displacement_time__x.onnx; matches the Python engine)
```

Force on a charge in an electric field — solve for electric field strength at the charge — `electric_force_in_field.Ef`

$$F = qE_f$$

input	meaning	unit
`F`	magnitude of the electric force	`N`
`q`	magnitude of the charge	`C`

- **Output unit:** `N/C`
- **File:** `electric_force_in_field__Ef.onnx`
- **Applies when:** electric

```
inputs {'F': 2.0, 'q': 2.1}
-> 0.952381 N/C   (ran on electric_force_in_field__Ef.onnx; matches the Python engine)
```

Force on a charge in an electric field — solve for magnitude of the electric force — `electric_force_in_field.F`

$$F = qE_f$$

input	meaning	unit
`q`	magnitude of the charge	`C`

input	meaning	unit
`Ef`	electric field strength at the charge	`N/C`

- **Output unit:** `N`
- **File:** `electric_force_in_field__F.onnx`
- **Applies when:** electric

```
inputs {'q': 2.0, 'Ef': 2.1}
-> 4.2 N   (ran on electric_force_in_field__F.onnx; matches the Python engine)
```

Force on a charge in an electric field — solve for magnitude of the charge — `electric_force_in_field.q`

$$F = qE_f$$

input	meaning	unit
`F`	magnitude of the electric force	`N`
`Ef`	electric field strength at the charge	`N/C`

- **Output unit:** `C`
- **File:** `electric_force_in_field__q.onnx`
- **Applies when:** electric

```
inputs {'F': 2.0, 'Ef': 2.1}
-> 0.952381 C   (ran on electric_force_in_field__q.onnx; matches the Python engine)
```

Electric power — solve for current — `electric_power.I`

$$P = VI$$

input	meaning	unit
`P`	electric power	`W`
`V`	voltage	`V`

- **Output unit:** `A`
- **File:** `electric_power__I.onnx`

```
inputs {'P': 2.0, 'V': 2.1}
-> 0.952381 A   (ran on electric_power__I.onnx; matches the Python engine)
```

Electric power — solve for electric power — `electric_power.P`

$$P = VI$$

input	meaning	unit
`V`	voltage	`V`
`I`	current	`A`

- **Output unit:** `W`
- **File:** `electric_power__P.onnx`

```
inputs {'V': 2.0, 'I': 2.1}
-> 4.2 W   (ran on electric_power__P.onnx; matches the Python engine)
```

Electric power — solve for voltage — `electric_power.V`

$$P = VI$$

input	meaning	unit
`P`	electric power	`W`
`I`	current	`A`

- **Output unit:** `V`
- **File:** `electric_power__V.onnx`

```
inputs {'P': 2.0, 'I': 2.1}
-> 0.952381 V   (ran on electric_power__V.onnx; matches the Python engine)
```

Efficiency of a heat engine — solve for heat taken in from the hot reservoir — `engine_efficiency.Qh`

$$\text{eff} = \frac{W}{Q_h}$$

input	meaning	unit
`eff`	efficiency (a fraction)	`1`
`W`	work output per cycle	`J`

- **Output unit:** `J`
- **File:** `engine_efficiency__Qh.onnx`
- **Applies when:** heat_engine

```
inputs {'eff': 2.0, 'W': 2.1}
-> 1.05 J   (ran on engine_efficiency__Qh.onnx; matches the Python engine)
```

Efficiency of a heat engine — solve for work output per cycle — `engine_efficiency.W`

$$\text{eff} = \frac{W}{Q_h}$$

input	meaning	unit
`eff`	efficiency (a fraction)	`1`
`Qh`	heat taken in from the hot reservoir	`J`

- **Output unit:** `J`
- **File:** `engine_efficiency__W.onnx`
- **Applies when:** heat_engine

```
inputs {'eff': 2.0, 'Qh': 2.1}
-> 4.2 J   (ran on engine_efficiency__W.onnx; matches the Python engine)
```

Efficiency of a heat engine — solve for efficiency (a fraction) — `engine_efficiency.eff`

$$\text{eff} = \frac{W}{Q_h}$$

input	meaning	unit
`W`	work output per cycle	`J`
`Qh`	heat taken in from the hot reservoir	`J`

- **Output unit:** `1`
- **File:** `engine_efficiency__eff.onnx`
- **Applies when:** heat_engine

```
inputs {'W': 2.0, 'Qh': 2.1}
-> 0.952381 1   (ran on engine_efficiency__eff.onnx; matches the Python engine)
```

Energy balance of a cyclic heat engine — solve for heat out to the cold reservoir — `engine_energy_balance.Qc`

$$W = Q_h - Q_c$$

input	meaning	unit
`W`	net work output per cycle	`J`
`Qh`	heat in from the hot reservoir	`J`

- **Output unit:** `J`
- **File:** `engine_energy_balance__Qc.onnx`
- **Applies when:** heat_engine

```
inputs {'W': 2.0, 'Qh': 2.1}
-> 0.1 J   (ran on engine_energy_balance__Qc.onnx; matches the Python engine)
```

Energy balance of a cyclic heat engine — solve for heat in from the hot reservoir — `engine_energy_balance.Qh`

$$W = Q_h - Q_c$$

input	meaning	unit
`W`	net work output per cycle	`J`
`Qc`	heat out to the cold reservoir	`J`

- **Output unit:** `J`
- **File:** `engine_energy_balance__Qh.onnx`
- **Applies when:** heat_engine

```
inputs {'W': 2.0, 'Qc': 2.1}
-> 4.1 J   (ran on engine_energy_balance__Qh.onnx; matches the Python engine)
```

Energy balance of a cyclic heat engine — solve for net work output per cycle — `engine_energy_balance.W`

$$W = Q_h - Q_c$$

input	meaning	unit
`Qh`	heat in from the hot reservoir	`J`
`Qc`	heat out to the cold reservoir	`J`

- **Output unit:** `J`
- **File:** `engine_energy_balance__W.onnx`
- **Applies when:** heat_engine

Flow rate through a cross section — solve for cross-sectional area — `flow_rate_area_speed.A`

$$Q = Av$$

input	meaning	unit
`Q`	volume flow rate	`m^3/s`
`v`	average speed of the fluid	`m/s`

- **Output unit:** `m^2`
- **File:** `flow_rate_area_speed__A.onnx`
- **Applies when:** flow

```
inputs {'Q': 2.0, 'v': 2.1}
-> 0.952381 m^2   (ran on flow_rate_area_speed__A.onnx; matches the Python engine)
```

Flow rate through a cross section — solve for volume flow rate — `flow_rate_area_speed.Q`

$$Q = Av$$

input	meaning	unit
`A`	cross-sectional area	`m^2`
`v`	average speed of the fluid	`m/s`

- **Output unit:** `m^3/s`
- **File:** `flow_rate_area_speed__Q.onnx`
- **Applies when:** flow

```
inputs {'A': 2.0, 'v': 2.1}
-> 4.2 m^3/s   (ran on flow_rate_area_speed__Q.onnx; matches the Python engine)
```

Flow rate through a cross section — solve for average speed of the fluid — `flow_rate_area_speed.v`

$$Q = Av$$

input	meaning	unit
`Q`	volume flow rate	`m^3/s`
`A`	cross-sectional area	`m^2`

- **Output unit:** `m/s`
- **File:** `flow_rate_area_speed__v.onnx`
- **Applies when:** flow

```
inputs {'Q': 2.0, 'A': 2.1}
-> 0.952381 m/s   (ran on flow_rate_area_speed__v.onnx; matches the Python engine)
```

Flow rate as volume per time — solve for volume flow rate — `flow_rate_volume_time.Q`

$$Q = \frac{V}{t}$$

input	meaning	unit
`V`	volume delivered	`m^3`
`t`	time taken	`s`

- **Output unit:** `m^3/s`
- **File:** `flow_rate_volume_time__Q.onnx`
- **Applies when:** flow

```
inputs {'V': 2.0, 't': 2.1}
-> 0.952381 m^3/s   (ran on flow_rate_volume_time__Q.onnx; matches the Python engine)
```

Flow rate as volume per time — solve for volume delivered — `flow_rate_volume_time.V`

$$Q = \frac{V}{t}$$

input	meaning	unit
`Q`	volume flow rate	`m^3/s`
`t`	time taken	`s`

- **Output unit:** `m^3`
- **File:** `flow_rate_volume_time__V.onnx`
- **Applies when:** flow

```
inputs {'Q': 2.0, 't': 2.1}
-> 4.2 m^3   (ran on flow_rate_volume_time__V.onnx; matches the Python engine)
```

Flow rate as volume per time — solve for time taken — `flow_rate_volume_time.t`

$$Q = \frac{V}{t}$$

input	meaning	unit
`Q`	volume flow rate	`m^3/s`
`V`	volume delivered	`m^3`

- **Output unit:** `s`
- **File:** `flow_rate_volume_time__t.onnx`
- **Applies when:** flow

```
inputs {'Q': 2.0, 'V': 2.1}
-> 1.05 s   (ran on flow_rate_volume_time__t.onnx; matches the Python engine)
```

Gravitational potential energy near the surface — solve for gravitational potential energy — `gravitational_pe.U`

$$U = mgh$$

input	meaning	unit
`m`	mass	`kg`
`g`	gravitational acceleration	`m/s^2`
`h`	height above the reference level	`m`

- **Output unit:** `J`
- **File:** `gravitational_pe__U.onnx`

```
inputs {'m': 2.0, 'g': 2.1, 'h': 2.2}
-> 9.24 J   (ran on gravitational_pe__U.onnx; matches the Python engine)
```

Gravitational potential energy near the surface — solve for gravitational acceleration — `gravitational_pe.g`

$$U = mgh$$

input	meaning	unit
`U`	gravitational potential energy	`J`
`m`	mass	`kg`
`h`	height above the reference level	`m`

- **Output unit:** `m/s^2`
- **File:** `gravitational_pe__g.onnx`

```
inputs {'U': 2.0, 'm': 2.1, 'h': 2.2}
-> 0.4329 m/s^2   (ran on gravitational_pe__g.onnx; matches the Python engine)
```

Gravitational potential energy near the surface — solve for height above the reference level — `gravitational_pe.h`

$$U = mgh$$

input	meaning	unit
`U`	gravitational potential energy	`J`
`m`	mass	`kg`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `gravitational_pe__h.onnx`

```
inputs {'U': 2.0, 'm': 2.1, 'g': 2.2}
-> 0.4329 m   (ran on gravitational_pe__h.onnx; matches the Python engine)
```

Gravitational potential energy near the surface — solve for mass — `gravitational_pe.m`

$$U = mgh$$

input	meaning	unit
`U`	gravitational potential energy	`J`

input	meaning	unit
`g`	gravitational acceleration	`m/s^2`
`h`	height above the reference level	`m`

- **Output unit:** `kg`
- **File:** `gravitational_pe__m.onnx`

```
inputs {'U': 2.0, 'g': 2.1, 'h': 2.2}
-> 0.4329 kg   (ran on gravitational_pe__m.onnx; matches the Python engine)
```

Heat for a temperature change — solve for heat transferred — `heat.Q`

$$Q = mc\Delta T$$

input	meaning	unit
`m`	mass	`kg`
`c`	specific heat capacity	`J/(kg*K)`
`dT`	temperature change	`K`

- **Output unit:** `J`
- **File:** `heat__Q.onnx`

```
inputs {'m': 2.0, 'c': 2.1, 'dT': 2.2}
-> 9.24 J   (ran on heat__Q.onnx; matches the Python engine)
```

Heat for a temperature change — solve for specific heat capacity — `heat.c`

$$Q = mc\Delta T$$

input	meaning	unit
`Q`	heat transferred	`J`
`m`	mass	`kg`
`dT`	temperature change	`K`

- **Output unit:** `J/(kg*K)`
- **File:** `heat__c.onnx`

```
inputs {'Q': 2.0, 'm': 2.1, 'dT': 2.2}
-> 0.4329 J/(kg*K)   (ran on heat__c.onnx; matches the Python engine)
```

Heat for a temperature change — solve for temperature change — `heat.dT`

$$Q = mc\Delta T$$

input	meaning	unit
`Q`	heat transferred	`J`
`m`	mass	`kg`
`c`	specific heat capacity	`J/(kg*K)`

- **Output unit:** `K`
- **File:** `heat__dT.onnx`

```
inputs {'Q': 2.0, 'm': 2.1, 'c': 2.2}
-> 0.4329 K   (ran on heat__dT.onnx; matches the Python engine)
```

Heat for a temperature change — solve for mass — `heat.m`

$$Q = mc\Delta T$$

input	meaning	unit
`Q`	heat transferred	`J`
`c`	specific heat capacity	`J/(kg*K)`
`dT`	temperature change	`K`

- **Output unit:** `kg`
- **File:** `heat__m.onnx`

```
inputs {'Q': 2.0, 'c': 2.1, 'dT': 2.2}
-> 0.4329 kg   (ran on heat__m.onnx; matches the Python engine)
```

Pressure of a column of fluid — solve for gauge pressure — `hydrostatic_pressure.P`

$$P = \rho g h$$

input	meaning	unit
`rho`	fluid density	`kg/m^3`
`g`	gravitational acceleration	`m/s^2`
`h`	depth below the surface	`m`

- **Output unit:** `Pa`
- **File:** `hydrostatic_pressure__P.onnx`

```
inputs {'rho': 2.0, 'g': 2.1, 'h': 2.2}
-> 9.24 Pa   (ran on hydrostatic_pressure__P.onnx; matches the Python engine)
```

Pressure of a column of fluid — solve for gravitational acceleration — `hydrostatic_pressure.g`

$$P = \rho g h$$

input	meaning	unit
`P`	gauge pressure	`Pa`
`rho`	fluid density	`kg/m^3`
`h`	depth below the surface	`m`

- **Output unit:** `m/s^2`
- **File:** `hydrostatic_pressure__g.onnx`

```
inputs {'P': 2.0, 'rho': 2.1, 'h': 2.2}
-> 0.4329 m/s^2   (ran on hydrostatic_pressure__g.onnx; matches the Python engine)
```

Pressure of a column of fluid — solve for depth below the surface — `hydrostatic_pressure.h`

$$P = \rho g h$$

input	meaning	unit
`P`	gauge pressure	`Pa`
`rho`	fluid density	`kg/m^3`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `hydrostatic_pressure__h.onnx`

```
inputs {'P': 2.0, 'rho': 2.1, 'g': 2.2}
-> 0.4329 m   (ran on hydrostatic_pressure__h.onnx; matches the Python engine)
```

Pressure of a column of fluid — solve for fluid density — `hydrostatic_pressure.rho`

$$P = \rho g h$$

input	meaning	unit
`P`	gauge pressure	`Pa`
`g`	gravitational acceleration	`m/s^2`
`h`	depth below the surface	`m`

- **Output unit:** `kg/m^3`
- **File:** `hydrostatic_pressure__rho.onnx`

```
inputs {'P': 2.0, 'g': 2.1, 'h': 2.2}
-> 0.4329 kg/m^3   (ran on hydrostatic_pressure__rho.onnx; matches the Python engine)
```

Ideal gas law — solve for absolute pressure — `ideal_gas_moles.P`

$$PV = nRT$$

input	meaning	unit
`V`	volume	`m^3`
`n`	amount of gas	`mol`
`T`	absolute temperature	`K`
`R`	gas constant	`J/(mol*K)`

- **Output unit:** `Pa`
- **File:** `ideal_gas_moles__P.onnx`
- **Applies when:** gas

```
inputs {'V': 2.0, 'n': 2.1, 'T': 2.2, 'R': 2.3}
-> 5.313 Pa   (ran on ideal_gas_moles__P.onnx; matches the Python engine)
```

Ideal gas law — solve for constant — `ideal_gas_moles.R`

$$PV = nRT$$

input	meaning	unit
`P`	absolute pressure	`Pa`
`V`	volume	`m^3`
`n`	amount of gas	`mol`
`T`	absolute temperature	`K`

- **Output unit:** `J/(mol*K)`
- **File:** `ideal_gas_moles__R.onnx`
- **Applies when:** gas

```
inputs {'P': 2.0, 'V': 2.1, 'n': 2.2, 'T': 2.3}
-> 0.83004 J/(mol*K)   (ran on ideal_gas_moles__R.onnx; matches the Python engine)
```

Ideal gas law — solve for absolute temperature — `ideal_gas_moles.T`

$$PV = nRT$$

input	meaning	unit
`P`	absolute pressure	`Pa`
`V`	volume	`m^3`
`n`	amount of gas	`mol`
`R`	gas constant	`J/(mol*K)`

- **Output unit:** `K`
- **File:** `ideal_gas_moles__T.onnx`
- **Applies when:** gas

```
inputs {'P': 2.0, 'V': 2.1, 'n': 2.2, 'R': 2.3}
-> 0.83004 K   (ran on ideal_gas_moles__T.onnx; matches the Python engine)
```

Ideal gas law — solve for volume — `ideal_gas_moles.V`

$$PV = nRT$$

input	meaning	unit
`P`	absolute pressure	`Pa`
`n`	amount of gas	`mol`
`T`	absolute temperature	`K`
`R`	gas constant	`J/(mol*K)`

- **Output unit:** `m^3`
- **File:** `ideal_gas_moles__V.onnx`
- **Applies when:** gas

```
inputs {'P': 2.0, 'n': 2.1, 'T': 2.2, 'R': 2.3}
-> 5.313 m^3   (ran on ideal_gas_moles__V.onnx; matches the Python engine)
```

Ideal gas law — solve for amount of gas — `ideal_gas_moles.n`

$$PV = nRT$$

input	meaning	unit
`P`	absolute pressure	`Pa`
`V`	volume	`m^3`
`T`	absolute temperature	`K`
`R`	gas constant	`J/(mol*K)`

- **Output unit:** `mol`
- **File:** `ideal_gas_moles__n.onnx`
- **Applies when:** gas

```
inputs {'P': 2.0, 'V': 2.1, 'T': 2.2, 'R': 2.3}
-> 0.83004 mol   (ran on ideal_gas_moles__n.onnx; matches the Python engine)
```

Impulse of a constant force — solve for force — `impulse.F`

$$J = Ft$$

input	meaning	unit
`J`	impulse	`N*s`
`t`	time the force acts	`s`

- **Output unit:** `N`
- **File:** `impulse__F.onnx`

```
inputs {'J': 2.0, 't': 2.1}
-> 0.952381 N   (ran on impulse__F.onnx; matches the Python engine)
```

Impulse of a constant force — solve for impulse — `impulse.J`

$$J = Ft$$

input	meaning	unit
`F`	force	`N`
`t`	time the force acts	`s`

- **Output unit:** `N*s`
- **File:** `impulse__J.onnx`

```
inputs {'F': 2.0, 't': 2.1}
-> 4.2 N*s   (ran on impulse__J.onnx; matches the Python engine)
```

Impulse of a constant force — solve for time the force acts — `impulse.t`

$$J = Ft$$

input	meaning	unit
`J`	impulse	`N*s`
`F`	force	`N`

- **Output unit:** `s`
- **File:** `impulse__t.onnx`

```
inputs {'J': 2.0, 'F': 2.1}
-> 0.952381 s   (ran on impulse__t.onnx; matches the Python engine)
```

Index of refraction — solve for light — `index_of_refraction.c`

$$n = \frac{c}{v}$$

input	meaning	unit
`n`	index of refraction	`1`
`v`	speed of light in the medium	`m/s`

- **Output unit:** `m/s`
- **File:** `index_of_refraction__c.onnx`
- **Applies when:** medium

```
inputs {'n': 2.0, 'v': 2.1}
-> 4.2 m/s   (ran on index_of_refraction__c.onnx; matches the Python engine)
```

Index of refraction — solve for index of refraction — `index_of_refraction.n`

$$n = \frac{c}{v}$$

input	meaning	unit
`c`	speed	`m/s`
`v`	speed of light in the medium	`m/s`

- **Output unit:** `1`
- **File:** `index_of_refraction__n.onnx`
- **Applies when:** medium

```
inputs {'c': 2.0, 'v': 2.1}
-> 0.952381 1   (ran on index_of_refraction__n.onnx; matches the Python engine)
```

Index of refraction — solve for speed of light in the medium — `index_of_refraction.v`

$$n = \frac{c}{v}$$

input	meaning	unit
`n`	index of refraction	`1`
`c`	speed	`m/s`

- **Output unit:** `m/s`
- **File:** `index_of_refraction__v.onnx`
- **Applies when:** medium

```
inputs {'n': 2.0, 'c': 2.1}
-> 1.05 m/s   (ran on index_of_refraction__v.onnx; matches the Python engine)
```

Reactance of an inductor — solve for inductance — `inductive_reactance.L`

$$X = 2\pi f L$$

input	meaning	unit
`X`	inductive reactance	`ohm`
`f`	frequency	`Hz`

- **Output unit:** `H`
- **File:** `inductive_reactance__L.onnx`
- **Applies when:** inductor

```
inputs {'X': 2.0, 'f': 2.1}
-> 0.151576 H   (ran on inductive_reactance__L.onnx; matches the Python engine)
```

Reactance of an inductor — solve for inductive reactance — `inductive_reactance.X`

$$X = 2\pi f L$$

input	meaning	unit
`f`	frequency	`Hz`
`L`	inductance	`H`

- **Output unit:** `ohm`
- **File:** `inductive_reactance__X.onnx`
- **Applies when:** inductor

```
inputs {'f': 2.0, 'L': 2.1}
-> 26.3894 ohm   (ran on inductive_reactance__X.onnx; matches the Python engine)
```

Reactance of an inductor — solve for frequency — `inductive_reactance.f`

$$X = 2\pi f L$$

input	meaning	unit
`X`	inductive reactance	`ohm`
`L`	inductance	`H`

- **Output unit:** `Hz`
- **File:** `inductive_reactance__f.onnx`
- **Applies when:** inductor

```
inputs {'X': 2.0, 'L': 2.1}
-> 0.151576 Hz   (ran on inductive_reactance__f.onnx; matches the Python engine)
```

Intensity as power per area — solve for area perpendicular to the flow of energy — `intensity_power_area.A`

$$I = \frac{P}{A}$$

input	meaning	unit
`I`	intensity	`W/m^2`
`P`	power crossing the area	`W`

- **Output unit:** `m^2`
- **File:** `intensity_power_area__A.onnx`

- **Applies when:** radiation

```
inputs {'I': 2.0, 'P': 2.1}
-> 1.05 m^2   (ran on intensity_power_area__A.onnx; matches the Python engine)
```

Intensity as power per area — solve for intensity — `intensity_power_area.I`

$$I = \frac{P}{A}$$

input	meaning	unit
`P`	power crossing the area	`W`
`A`	area perpendicular to the flow of energy	`m^2`

- **Output unit:** `W/m^2`
- **File:** `intensity_power_area__I.onnx`
- **Applies when:** radiation

```
inputs {'P': 2.0, 'A': 2.1}
-> 0.952381 W/m^2   (ran on intensity_power_area__I.onnx; matches the Python engine)
```

Intensity as power per area — solve for power crossing the area — `intensity_power_area.P`

$$I = \frac{P}{A}$$

input	meaning	unit
`I`	intensity	`W/m^2`
`A`	area perpendicular to the flow of energy	`m^2`

- **Output unit:** `W`
- **File:** `intensity_power_area__P.onnx`
- **Applies when:** radiation

```
inputs {'I': 2.0, 'A': 2.1}
-> 4.2 W   (ran on intensity_power_area__P.onnx; matches the Python engine)
```

Translational kinetic energy — solve for kinetic energy — `kinetic_energy.K`

$$K = \frac{mv^2}{2}$$

input	meaning	unit
`m`	mass	`kg`
`v`	speed	`m/s`

- **Output unit:** `J`
- **File:** `kinetic_energy__K.onnx`

```
inputs {'m': 2.0, 'v': 2.1}
-> 4.41 J   (ran on kinetic_energy__K.onnx; matches the Python engine)
```

Translational kinetic energy — solve for mass — `kinetic_energy.m`

$$K = \frac{mv^2}{2}$$

input	meaning	unit
`K`	kinetic energy	`J`
`v`	speed	`m/s`

- **Output unit:** `kg`
- **File:** `kinetic_energy__m.onnx`

```
inputs {'K': 2.0, 'v': 2.1}
-> 0.907029 kg   (ran on kinetic_energy__m.onnx; matches the Python engine)
```

Translational kinetic energy — solve for speed — `kinetic_energy.v`

$$K = \frac{mv^2}{2}$$

input	meaning	unit
`K`	kinetic energy	`J`
`m`	mass	kg

- **Output unit:** `m/s`
- **File:** `kinetic_energy__v.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'K': 2.0, 'm': 2.1}
-> -1.38013, 1.38013 m/s   (ran on kinetic_energy__v.onnx; matches the Python engine)
```

Sliding friction — solve for normal force — `kinetic_friction.N`

$$f = \mu N$$

input	meaning	unit
`f`	friction force	`N`
`mu`	coefficient of friction	`1`

- **Output unit:** `N`

- **File:** `kinetic_friction__N.onnx`

```
inputs {'f': 2.0, 'mu': 2.1}
-> 0.952381 N   (ran on kinetic_friction__N.onnx; matches the Python engine)
```

Sliding friction — solve for friction force — `kinetic_friction.f`

$$f = \mu N$$

input	meaning	unit
`mu`	coefficient of friction	`1`
`N`	normal force	`N`

- **Output unit:** `N`
- **File:** `kinetic_friction__f.onnx`

```
inputs {'mu': 2.0, 'N': 2.1}
-> 4.2 N   (ran on kinetic_friction__f.onnx; matches the Python engine)
```

Sliding friction — solve for coefficient of friction — `kinetic_friction.mu`

$$f = \mu N$$

input	meaning	unit
`f`	friction force	`N`
`N`	normal force	`N`

- **Output unit:** `1`
- **File:** `kinetic_friction__mu.onnx`

```
inputs {'f': 2.0, 'N': 2.1}
-> 0.952381 1   (ran on kinetic_friction__mu.onnx; matches the Python engine)
```

Heat to melt or freeze — solve for latent heat of fusion — `latent_heat_fusion.Lf`

$$Q = mL_f$$

input	meaning	unit
`Q`	heat absorbed or released	`J`
`m`	mass	kg

- **Output unit:** `J/kg`
- **File:** `latent_heat_fusion__Lf.onnx`
- **Applies when:** melting

```
inputs {'Q': 2.0, 'm': 2.1}
-> 0.952381 J/kg   (ran on latent_heat_fusion__Lf.onnx; matches the Python engine)
```

Heat to melt or freeze — solve for heat absorbed or released — `latent_heat_fusion.Q`

$$Q = mL_f$$

input	meaning	unit
`m`	mass	`kg`
`Lf`	latent heat of fusion	`J/kg`

- **Output unit:** `J`
- **File:** `latent_heat_fusion__Q.onnx`
- **Applies when:** melting

```
inputs {'m': 2.0, 'Lf': 2.1}
-> 4.2 J   (ran on latent_heat_fusion__Q.onnx; matches the Python engine)
```

Heat to melt or freeze — solve for mass — `latent_heat_fusion.m`

$$Q = mL_f$$

input	meaning	unit
`Q`	heat absorbed or released	`J`
`Lf`	latent heat of fusion	`J/kg`

- **Output unit:** `kg`
- **File:** `latent_heat_fusion__m.onnx`
- **Applies when:** melting

```
inputs {'Q': 2.0, 'Lf': 2.1}
-> 0.952381 kg   (ran on latent_heat_fusion__m.onnx; matches the Python engine)
```

Heat to boil or condense — solve for latent heat of vaporization — `latent_heat_vaporization.Lv`

$$Q = mL_v$$

input	meaning	unit
`Q`	energy	`J`
`m`	mass	`kg`

- **Output unit:** `J/kg`
- **File:** `latent_heat_vaporization__Lv.onnx`
- **Applies when:** boiling

```
inputs {'Q': 2.0, 'm': 2.1}
-> 0.952381 J/kg   (ran on latent_heat_vaporization__Lv.onnx; matches the Python engine)
```

Heat to boil or condense — solve for heat — `latent_heat_vaporization.Q`

$$Q = mL_v$$

input	meaning	unit
`m`	mass	`kg`
`Lv`	latent heat of vaporization	`J/kg`

- **Output unit:** `J`
- **File:** `latent_heat_vaporization__Q.onnx`
- **Applies when:** boiling

```
inputs {'m': 2.0, 'Lv': 2.1}
-> 4.2 J   (ran on latent_heat_vaporization__Q.onnx; matches the Python engine)
```

Heat to boil or condense — solve for mass — `latent_heat_vaporization.m`

$$Q = mL_v$$

input	meaning	unit
`Q`	energy	`J`
`Lv`	latent heat of vaporization	`J/kg`

- **Output unit:** `kg`
- **File:** `latent_heat_vaporization__m.onnx`
- **Applies when:** boiling

```
inputs {'Q': 2.0, 'Lv': 2.1}
-> 0.952381 kg   (ran on latent_heat_vaporization__m.onnx; matches the Python engine)
```

Resonant frequency of an LC circuit — solve for capacitance — `lc_resonance.C`

$$f = \frac{1}{2\pi\sqrt{LC}}$$

input	meaning	unit
`f`	resonant frequency	`Hz`
`L`	inductance	`H`

- **Output unit:** `F`
- **File:** `lc_resonance__C.onnx`
- **Applies when:** lc_circuit

```
inputs {'f': 2.0, 'L': 2.1}
-> 0.00301551 F   (ran on lc_resonance__C.onnx; matches the Python engine)
```

Resonant frequency of an LC circuit — solve for inductance — `lc_resonance.L`

$$f = \frac{1}{2\pi\sqrt{LC}}$$

input	meaning	unit
`f`	resonant frequency	`Hz`
`C`	capacitance	`F`

- **Output unit:** `H`
- **File:** `lc_resonance__L.onnx`
- **Applies when:** lc_circuit

```
inputs {'f': 2.0, 'C': 2.1}
-> 0.00301551 H   (ran on lc_resonance__L.onnx; matches the Python engine)
```

Resonant frequency of an LC circuit — solve for resonant frequency — `lc_resonance.f`

$$f = \frac{1}{2\pi\sqrt{LC}}$$

input	meaning	unit
`L`	inductance	`H`
`C`	capacitance	`F`

- **Output unit:** `Hz`
- **File:** `lc_resonance__f.onnx`
- **Applies when:** lc_circuit

```
inputs {'L': 2.0, 'C': 2.1}
-> 0.0776597 Hz   (ran on lc_resonance__f.onnx; matches the Python engine)
```

Linear thermal expansion — solve for original length — `linear_expansion.L`

$$d_L = \alpha_t L d_T$$

input	meaning	unit
`dL`	change in length	`m`
`alpha_t`	coefficient of linear expansion	`1/K`
`dT`	change in temperature	`K`

- **Output unit:** `m`

- **File:** `linear_expansion__L.onnx`
- **Applies when:** thermal_expansion

```
inputs {'dL': 2.0, 'alpha_t': 2.1, 'dT': 2.2}
-> 0.4329 m   (ran on linear_expansion__L.onnx; matches the Python engine)
```

Linear thermal expansion — solve for coefficient of linear expansion — `linear_expansion.alpha_t`

$$d_L = \alpha_t L d_T$$

input	meaning	unit
`dL`	change in length	`m`
`L`	original length	`m`
`dT`	change in temperature	`K`

- **Output unit:** `1/K`
- **File:** `linear_expansion__alpha_t.onnx`
- **Applies when:** thermal_expansion

```
inputs {'dL': 2.0, 'L': 2.1, 'dT': 2.2}
-> 0.4329 1/K   (ran on linear_expansion__alpha_t.onnx; matches the Python engine)
```

Linear thermal expansion — solve for change in length — `linear_expansion.dL`

$$d_L = \alpha_t L d_T$$

input	meaning	unit
`alpha_t`	coefficient of linear expansion	`1/K`
`L`	original length	`m`
`dT`	change in temperature	`K`

- **Output unit:** `m`
- **File:** `linear_expansion__dL.onnx`
- **Applies when:** thermal_expansion

```
inputs {'alpha_t': 2.0, 'L': 2.1, 'dT': 2.2}
-> 9.24 m   (ran on linear_expansion__dL.onnx; matches the Python engine)
```

Linear thermal expansion — solve for change in temperature — `linear_expansion.dT`

$$d_L = \alpha_t L d_T$$

input	meaning	unit
`dL`	change in length	`m`
`alpha_t`	coefficient of linear expansion	`1/K`
`L`	original length	`m`

- **Output unit:** `K`
- **File:** `linear_expansion__dT.onnx`
- **Applies when:** thermal_expansion

```
inputs {'dL': 2.0, 'alpha_t': 2.1, 'L': 2.2}
-> 0.4329 K   (ran on linear_expansion__dT.onnx; matches the Python engine)
```

Magnetic field of a long straight wire — solve for magnetic field — `long_wire_field.B`

$$B = \frac{\mu_0 I}{2\pi r}$$

input	meaning	unit
`I`	current	`A`
`r`	distance from the wire	`m`
`mu0`	permeability	`T*m/A`

- **Output unit:** `T`
- **File:** `long_wire_field__B.onnx`
- **Applies when:** wire

```
inputs {'I': 2.0, 'r': 2.1, 'mu0': 2.2}
-> 0.333467 T   (ran on long_wire_field__B.onnx; matches the Python engine)
```

Magnetic field of a long straight wire — solve for current — `long_wire_field.I`

$$B = \frac{\mu_0 I}{2\pi r}$$

input	meaning	unit
`B`	magnetic field	`T`
`r`	distance from the wire	`m`
`mu0`	permeability	`T*m/A`

- **Output unit:** `A`
- **File:** `long_wire_field__I.onnx`
- **Applies when:** wire

```
inputs {'B': 2.0, 'r': 2.1, 'mu0': 2.2}
-> 11.9952 A   (ran on long_wire_field__I.onnx; matches the Python engine)
```

Magnetic field of a long straight wire — solve for constant — `long_wire_field.mu0`

$$B = \frac{\mu_0 I}{2\pi r}$$

input	meaning	unit
`B`	magnetic field	`T`
`I`	current	`A`
`r`	distance from the wire	`m`

- **Output unit:** `T*m/A`
- **File:** `long_wire_field__mu0.onnx`
- **Applies when:** wire

```
inputs {'B': 2.0, 'I': 2.1, 'r': 2.2}
-> 13.1648 T*m/A   (ran on long_wire_field__mu0.onnx; matches the Python engine)
```

Magnetic field of a long straight wire — solve for distance from the wire — `long_wire_field.r`

$$B = \frac{\mu_0 I}{2\pi r}$$

input	meaning	unit
`B`	magnetic field	`T`
`I`	current	`A`
`mu0`	permeability	`T*m/A`

- **Output unit:** `m`
- **File:** `long_wire_field__r.onnx`
- **Applies when:** wire

```
inputs {'B': 2.0, 'I': 2.1, 'mu0': 2.2}
-> 0.367648 m   (ran on long_wire_field__r.onnx; matches the Python engine)
```

Magnetic field at the centre of a circular loop — solve for field at the centre — `loop_center_field.B`

$$B = \frac{\mu_0 I}{2r}$$

input	meaning	unit
`I`	current	`A`
`r`	radius of the loop	`m`
`mu0`	permeability	`T*m/A`

- **Output unit:** `T`
- **File:** `loop_center_field__B.onnx`
- **Applies when:** loop

```
inputs {'I': 2.0, 'r': 2.1, 'mu0': 2.2}
-> 1.04762 T   (ran on loop_center_field__B.onnx; matches the Python engine)
```

Magnetic field at the centre of a circular loop — solve for current — `loop_center_field.I`

$$B = \frac{\mu_0 I}{2r}$$

input	meaning	unit
`B`	field at the centre	`T`
`r`	radius of the loop	`m`
`mu0`	permeability	`T*m/A`

- **Output unit:** `A`
- **File:** `loop_center_field__I.onnx`
- **Applies when:** loop

```
inputs {'B': 2.0, 'r': 2.1, 'mu0': 2.2}
-> 3.81818 A   (ran on loop_center_field__I.onnx; matches the Python engine)
```

Magnetic field at the centre of a circular loop — solve for constant — `loop_center_field.mu0`

$$B = \frac{\mu_0 I}{2r}$$

input	meaning	unit
`B`	field at the centre	`T`
`I`	current	`A`
`r`	radius of the loop	`m`

- **Output unit:** `T*m/A`
- **File:** `loop_center_field__mu0.onnx`
- **Applies when:** loop

```
inputs {'B': 2.0, 'I': 2.1, 'r': 2.2}
-> 4.19048 T*m/A   (ran on loop_center_field__mu0.onnx; matches the Python engine)
```

Magnetic field at the centre of a circular loop — solve for radius of the loop — `loop_center_field.r`

$$B = \frac{\mu_0 I}{2r}$$

input	meaning	unit
`B`	field at the centre	`T`
`I`	current	`A`
`mu0`	permeability	`T*m/A`

- **Output unit:** `m`
- **File:** `loop_center_field__r.onnx`
- **Applies when:** loop

```
inputs {'B': 2.0, 'I': 2.1, 'mu0': 2.2}
-> 1.155 m   (ran on loop_center_field__r.onnx; matches the Python engine)
```

Magnetic force on a moving charge — solve for magnetic field — `magnetic_force_charge.B`

$$F = qvB\sin(\theta)$$

input	meaning	unit
`F`	magnitude of the magnetic force	`N`
`q`	charge	`C`
`v`	speed of the charge	`m/s`
`theta`	angle between velocity and field	`rad`

- **Output unit:** `T`
- **File:** `magnetic_force_charge__B.onnx`
- **Applies when:** magnetic

```
inputs {'F': 2.0, 'q': 2.1, 'v': 2.2, 'theta': 2.3}
-> 0.580525 T   (ran on magnetic_force_charge__B.onnx; matches the Python engine)
```

Magnetic force on a moving charge — solve for magnitude of the magnetic force — `magnetic_force_charge.F`

$$F = qvB\sin(\theta)$$

input	meaning	unit
`q`	charge	`C`
`v`	speed of the charge	`m/s`
`B`	magnetic field	`T`
`theta`	angle between velocity and field	`rad`

- **Output unit:** `N`
- **File:** `magnetic_force_charge__F.onnx`
- **Applies when:** magnetic

```
inputs {'q': 2.0, 'v': 2.1, 'B': 2.2, 'theta': 2.3}
-> 6.89032 N   (ran on magnetic_force_charge__F.onnx; matches the Python engine)
```

Magnetic force on a moving charge — solve for charge — `magnetic_force_charge.q`

$$F = qvB\sin(\theta)$$

input	meaning	unit
`F`	magnitude of the magnetic force	`N`
`v`	speed of the charge	`m/s`
`B`	magnetic field	`T`
`theta`	angle between velocity and field	`rad`

- **Output unit:** `C`
- **File:** `magnetic_force_charge__q.onnx`
- **Applies when:** magnetic

```
inputs {'F': 2.0, 'v': 2.1, 'B': 2.2, 'theta': 2.3}
-> 0.580525 C   (ran on magnetic_force_charge__q.onnx; matches the Python engine)
```

Magnetic force on a moving charge — solve for angle between velocity and field — `magnetic_force_charge.theta`

$$F = qvB\sin(\theta)$$

input	meaning	unit
`F`	magnitude of the magnetic force	`N`
`q`	charge	`C`
`v`	speed of the charge	`m/s`
`B`	magnetic field	`T`

- **Output unit:** `rad`
- **File:** `magnetic_force_charge__theta.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** magnetic

Magnetic force on a moving charge — solve for speed of the charge — `magnetic_force_charge.v`

$$F = qvB\sin(\theta)$$

input	meaning	unit
`F`	magnitude of the magnetic force	`N`
`q`	charge	`C`

input	meaning	unit
`B`	magnetic field	`T`
`theta`	angle between velocity and field	`rad`

- **Output unit:** `m/s`
- **File:** `magnetic_force_charge__v.onnx`
- **Applies when:** magnetic

```
inputs {'F': 2.0, 'q': 2.1, 'B': 2.2, 'theta': 2.3}
-> 0.580525 m/s   (ran on magnetic_force_charge__v.onnx; matches the Python engine)
```

Magnetic force on a current-carrying wire — solve for magnetic field — `magnetic_force_wire.B`

$$F = ILB\sin\left(\theta\right)$$

input	meaning	unit
`F`	force	`N`
`I`	current	`A`
`L`	length of wire in the field	`m`
`theta`	angle between the wire and the field	`rad`

- **Output unit:** `T`
- **File:** `magnetic_force_wire__B.onnx`
- **Applies when:** magnetic; wire

```
inputs {'F': 2.0, 'I': 2.1, 'L': 2.2, 'theta': 2.3}
-> 0.580525 T   (ran on magnetic_force_wire__B.onnx; matches the Python engine)
```

Magnetic force on a current-carrying wire — solve for magnetic — `magnetic_force_wire.F`

$$F = ILB\sin\left(\theta\right)$$

input	meaning	unit
`I`	current	`A`
`L`	length of wire in the field	`m`
`B`	magnetic field	`T`
`theta`	angle between the wire and the field	`rad`

- **Output unit:** `N`
- **File:** `magnetic_force_wire__F.onnx`
- **Applies when:** magnetic; wire

```
inputs {'I': 2.0, 'L': 2.1, 'B': 2.2, 'theta': 2.3}
-> 6.89032 N   (ran on magnetic_force_wire__F.onnx; matches the Python engine)
```

Magnetic force on a current-carrying wire — solve for current — `magnetic_force_wire.I`

$$F = ILB\sin(\theta)$$

input	meaning	unit
`F`	force	`N`
`L`	length of wire in the field	`m`
`B`	magnetic field	`T`
`theta`	angle between the wire and the field	`rad`

- **Output unit:** `A`
- **File:** `magnetic_force_wire__I.onnx`
- **Applies when:** magnetic; wire

```
inputs {'F': 2.0, 'L': 2.1, 'B': 2.2, 'theta': 2.3}
-> 0.580525 A   (ran on magnetic_force_wire__I.onnx; matches the Python engine)
```

Magnetic force on a current-carrying wire — solve for length of wire in the field — `magnetic_force_wire.L`

$$F = ILB\sin(\theta)$$

input	meaning	unit
`F`	force	`N`
`I`	current	`A`
`B`	magnetic field	`T`
`theta`	angle between the wire and the field	`rad`

- **Output unit:** `m`
- **File:** `magnetic_force_wire__L.onnx`
- **Applies when:** magnetic; wire

```
inputs {'F': 2.0, 'I': 2.1, 'B': 2.2, 'theta': 2.3}
-> 0.580525 m   (ran on magnetic_force_wire__L.onnx; matches the Python engine)
```

Magnetic force on a current-carrying wire — solve for angle between the wire and the field — `magnetic_force_wire.theta`

$$F = ILB\sin(\theta)$$

input	meaning	unit
`F`	force	`N`
`I`	current	`A`
`L`	length of wire in the field	`m`
`B`	magnetic field	`T`

- **Output unit:** `rad`
- **File:** `magnetic_force_wire__theta.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** magnetic; wire

Maximum height of a vertical throw — solve for gravitational acceleration — `max_height_vertical_throw.g`

$$h = \frac{v_0^2}{2g}$$

input	meaning	unit
`h`	maximum height above the launch point	`m`
`v0`	launch speed	`m/s`

- **Output unit:** `m/s^2`
- **File:** `max_height_vertical_throw__g.onnx`
- **Applies when:** thrown_up

```
inputs {'h': 2.0, 'v0': 2.1}
-> 1.1025 m/s^2   (ran on max_height_vertical_throw__g.onnx; matches the Python engine)
```

Maximum height of a vertical throw — solve for maximum height above the launch point — `max_height_vertical_throw.h`

$$h = \frac{v_0^2}{2g}$$

input	meaning	unit
`v0`	launch speed	`m/s`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `max_height_vertical_throw__h.onnx`
- **Applies when:** thrown_up

```
inputs {'v0': 2.0, 'g': 2.1}
-> 0.952381 m   (ran on max_height_vertical_throw__h.onnx; matches the Python engine)
```

Maximum height of a vertical throw — solve for launch speed — `max_height_vertical_throw.v0`

$$h = \frac{v_0^2}{2g}$$

input	meaning	unit
`h`	maximum height above the launch point	`m`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `m/s`
- **File:** `max_height_vertical_throw__v0.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** thrown_up

```
inputs {'h': 2.0, 'g': 2.1}
-> -2.89828, 2.89828 m/s   (ran on max_height_vertical_throw__v0.onnx; matches the Python engine)
```

Focal length of a spherical mirror — solve for radius of curvature — `mirror_focal_length.R`

$$f = \frac{R}{2}$$

input	meaning	unit
`f`	focal length	`m`

- **Output unit:** `m`
- **File:** `mirror_focal_length__R.onnx`
- **Applies when:** mirror

```
inputs {'f': 2.0}
-> 4 m   (ran on mirror_focal_length__R.onnx; matches the Python engine)
```

Focal length of a spherical mirror — solve for focal length — `mirror_focal_length.f`

$$f = \frac{R}{2}$$

input	meaning	unit
`R`	radius of curvature	`m`

- **Output unit:** `m`
- **File:** `mirror_focal_length__f.onnx`
- **Applies when:** mirror

```
inputs {'R': 2.0}
-> 1 m   (ran on mirror_focal_length__f.onnx; matches the Python engine)
```

Linear momentum — solve for mass — `momentum.m`

$$p = mv$$

input	meaning	unit
`p`	momentum	`kg*m/s`
`v`	velocity	`m/s`

- **Output unit:** `kg`
- **File:** `momentum__m.onnx`

```
inputs {'p': 2.0, 'v': 2.1}
-> 0.952381 kg   (ran on momentum__m.onnx; matches the Python engine)
```

Linear momentum — solve for momentum — `momentum.p`

$$p = mv$$

input	meaning	unit
`m`	mass	`kg`
`v`	velocity	`m/s`

- **Output unit:** `kg*m/s`
- **File:** `momentum__p.onnx`

```
inputs {'m': 2.0, 'v': 2.1}
-> 4.2 kg*m/s   (ran on momentum__p.onnx; matches the Python engine)
```

Linear momentum — solve for velocity — `momentum.v`

$$p = mv$$

input	meaning	unit
`p`	momentum	`kg*m/s`
`m`	mass	`kg`

- **Output unit:** `m/s`
- **File:** `momentum__v.onnx`

```
inputs {'p': 2.0, 'm': 2.1}
-> 0.952381 m/s   (ran on momentum__v.onnx; matches the Python engine)
```

Emf of a rod moving through a magnetic field — solve for magnetic field — `motional_emf.B`

$$\mathrm{emf} = Blv$$

input	meaning	unit
`emf`	induced emf	`V`
`l`	length of the rod	`m`
`v`	speed of the rod	`m/s`

- **Output unit:** `T`
- **File:** `motional_emf__B.onnx`
- **Applies when:** moving_rod

```
inputs {'emf': 2.0, 'l': 2.1, 'v': 2.2}
-> 0.4329 T   (ran on motional_emf__B.onnx; matches the Python engine)
```

Emf of a rod moving through a magnetic field — solve for induced emf — `motional_emf.emf`

$$\mathrm{emf} = Blv$$

input	meaning	unit
`B`	magnetic field	`T`
`l`	length of the rod	`m`
`v`	speed of the rod	`m/s`

- **Output unit:** `V`
- **File:** `motional_emf__emf.onnx`
- **Applies when:** moving_rod

```
inputs {'B': 2.0, 'l': 2.1, 'v': 2.2}
-> 9.24 V   (ran on motional_emf__emf.onnx; matches the Python engine)
```

Emf of a rod moving through a magnetic field — solve for length of the rod — `motional_emf.l`

$$\mathrm{emf} = Blv$$

input	meaning	unit
`emf`	induced emf	`V`
`B`	magnetic field	`T`
`v`	speed of the rod	`m/s`

- **Output unit:** `m`
- **File:** `motional_emf__l.onnx`
- **Applies when:** moving_rod

```
inputs {'emf': 2.0, 'B': 2.1, 'v': 2.2}
-> 0.4329 m   (ran on motional_emf__l.onnx; matches the Python engine)
```

Emf of a rod moving through a magnetic field — solve for speed of the rod — `motional_emf.v`

$$\text{emf} = Blv$$

input	meaning	unit
`emf`	induced emf	`V`
`B`	magnetic field	`T`
`l`	length of the rod	`m`

- **Output unit:** `m/s`
- **File:** `motional_emf__v.onnx`
- **Applies when:** moving_rod

```
inputs {'emf': 2.0, 'B': 2.1, 'l': 2.2}
-> 0.4329 m/s   (ran on motional_emf__v.onnx; matches the Python engine)
```

Net force with an opposing friction force — solve for net force — `net_force_friction.F`

$$F = P - f$$

input	meaning	unit
`P`	applied force	`N`
`f`	friction force	`N`

- **Output unit:** `N`
- **File:** `net_force_friction__F_upper.onnx`

```
inputs {'P': 2.0, 'f': 2.1}
-> -0.1 N   (ran on net_force_friction__F_upper.onnx; matches the Python engine)
```

Net force with an opposing friction force — solve for applied force — `net_force_friction.P`

$$F = P - f$$

input	meaning	unit
`F`	net force	`N`
`f`	friction force	`N`

- **Output unit:** `N`
- **File:** `net_force_friction__P.onnx`

```
inputs {'F': 2.0, 'f': 2.1}
-> 4.1 N   (ran on net_force_friction__P.onnx; matches the Python engine)
```

Net force with an opposing friction force — solve for friction force — `net_force_friction.f`

$$F = P - f$$

input	meaning	unit
`F`	net force	`N`
`P`	applied force	`N`

- **Output unit:** `N`
- **File:** `net_force_friction__f.onnx`

```
inputs {'F': 2.0, 'P': 2.1}
-> 0.1 N   (ran on net_force_friction__f.onnx; matches the Python engine)
```

Work-energy theorem — solve for net work done on the body, equal to its change in kinetic energy — `net_work_kinetic_energy.W`

$$W = \frac{m\left(v^2 - v_0^2\right)}{2}$$

input	meaning	unit
`m`	mass	`kg`
`v`	final speed	`m/s`
`v0`	initial speed	`m/s`

- **Output unit:** `J`
- **File:** `net_work_kinetic_energy__W.onnx`

```
inputs {'m': 2.0, 'v': 2.1, 'v0': 2.2}
-> -0.43 J   (ran on net_work_kinetic_energy__W.onnx; matches the Python engine)
```

Work-energy theorem — solve for mass — `net_work_kinetic_energy.m`

$$W = \frac{m\left(v^2 - v_0^2\right)}{2}$$

input	meaning	unit
`W`	net work done on the body, equal to its change in kinetic energy	`J`
`v`	final speed	`m/s`
`v0`	initial speed	`m/s`

- **Output unit:** `kg`
- **File:** `net_work_kinetic_energy__m.onnx`

Work-energy theorem — solve for final speed — `net_work_kinetic_energy.v`

$$W = \frac{m\left(v^2 - v_0^2\right)}{2}$$

input	meaning	unit
`W`	net work done on the body, equal to its change in kinetic energy	`J`
`m`	mass	`kg`
`v0`	initial speed	`m/s`

- **Output unit:** `m/s`
- **File:** `net_work_kinetic_energy__v.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'W': 2.0, 'm': 2.1, 'v0': 2.2}
-> -2.59707, 2.59707 m/s   (ran on net_work_kinetic_energy__v.onnx; matches the Python engine)
```

Work-energy theorem — solve for initial speed — `net_work_kinetic_energy.v0`

$$W = \frac{m\left(v^2 - v_0^2\right)}{2}$$

input	meaning	unit
`W`	net work done on the body, equal to its change in kinetic energy	`J`
`m`	mass	`kg`
`v`	final speed	`m/s`

- **Output unit:** `m/s`
- **File:** `net_work_kinetic_energy__v0.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'W': 2.0, 'm': 2.1, 'v': 2.2}
-> -1.71325, 1.71325 m/s   (ran on net_work_kinetic_energy__v0.onnx; matches the Python engine)
```

Net force and acceleration — solve for net force — `newton_second.F`

$$F = ma$$

input	meaning	unit
`m`	mass	`kg`
`a`	acceleration	`m/s^2`

- **Output unit:** `N`
- **File:** `newton_second__F.onnx`

```
inputs {'m': 2.0, 'a': 2.1}
-> 4.2 N   (ran on newton_second__F.onnx; matches the Python engine)
```

Net force and acceleration — solve for acceleration — `newton_second.a`

$$F = ma$$

input	meaning	unit
`F`	net force	`N`
`m`	mass	kg

- **Output unit:** `m/s^2`
- **File:** `newton_second__a.onnx`

```
inputs {'F': 2.0, 'm': 2.1}
-> 0.952381 m/s^2   (ran on newton_second__a.onnx; matches the Python engine)
```

Net force and acceleration — solve for mass — `newton_second.m`

$$F = ma$$

input	meaning	unit
`F`	net force	`N`
`a`	acceleration	`m/s^2`

- **Output unit:** `kg`
- **File:** `newton_second__m.onnx`

```
inputs {'F': 2.0, 'a': 2.1}
-> 0.952381 kg   (ran on newton_second__m.onnx; matches the Python engine)
```

Normal force on an incline — solve for normal force from the incline — `normal_force_incline.N`

$$N = mg\cos\left(\theta\right)$$

input	meaning	unit
`m`	mass	`kg`
`g`	gravitational acceleration	`m/s^2`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `N`
- **File:** `normal_force_incline__N.onnx`
- **Applies when:** inclined

```
inputs {'m': 0.5, 'g': 0.6, 'theta': 0.7}
-> 0.229453 N   (ran on normal_force_incline__N.onnx; matches the Python engine)
```

Normal force on an incline — solve for gravitational acceleration — `normal_force_incline.g`

$$N = mg\cos\left(\theta\right)$$

input	meaning	unit
`N`	normal force from the incline	`N`
`m`	mass	`kg`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `m/s^2`
- **File:** `normal_force_incline__g.onnx`
- **Applies when:** inclined

```
inputs {'N': 0.5, 'm': 0.6, 'theta': 0.7}
-> 1.08955 m/s^2   (ran on normal_force_incline__g.onnx; matches the Python engine)
```

Normal force on an incline — solve for mass — `normal_force_incline.m`

$$N = mg\cos\left(\theta\right)$$

input	meaning	unit
`N`	normal force from the incline	`N`
`g`	gravitational acceleration	`m/s^2`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `kg`
- **File:** `normal_force_incline__m.onnx`
- **Applies when:** inclined

```
inputs {'N': 0.5, 'g': 0.6, 'theta': 0.7}
-> 1.08955 kg   (ran on normal_force_incline__m.onnx; matches the Python engine)
```

Normal force on an incline — solve for angle of the incline to the horizontal — `normal_force_incline.theta`

$$N = mg\cos\left(\theta\right)$$

input	meaning	unit
`N`	normal force from the incline	`N`
`m`	mass	`kg`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `rad`
- **File:** `normal_force_incline__theta.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** inclined

Normal force on a level surface — solve for normal force — `normal_on_level.N`

$$N = W$$

input	meaning	unit
`W`	weight	`N`

- **Output unit:** `N`
- **File:** `normal_on_level__N.onnx`

```
inputs {'W': 2.0}
-> 2 N   (ran on normal_on_level__N.onnx; matches the Python engine)
```

Normal force on a level surface — solve for weight — `normal_on_level.W`

$$N = W$$

input	meaning	unit
`N`	normal force	`N`

- **Output unit:** `N`
- **File:** `normal_on_level__W.onnx`

```
inputs {'N': 2.0}
-> 2 N   (ran on normal_on_level__W.onnx; matches the Python engine)
```

Voltage across an ohmic resistance — solve for current — `ohm.I`

$$V = IR$$

input	meaning	unit
`V`	voltage	`V`
`R`	resistance	`ohm`

- **Output unit:** `A`
- **File:** `ohm__I.onnx`

```
inputs {'V': 2.0, 'R': 2.1}
-> 0.952381 A   (ran on ohm__I.onnx; matches the Python engine)
```

Voltage across an ohmic resistance — solve for resistance — `ohm.R`

$$V = IR$$

input	meaning	unit
`V`	voltage	`V`
`I`	current	`A`

- **Output unit:** `ohm`
- **File:** `ohm__R.onnx`

```
inputs {'V': 2.0, 'I': 2.1}
-> 0.952381 ohm   (ran on ohm__R.onnx; matches the Python engine)
```

Voltage across an ohmic resistance — solve for voltage — `ohm.V`

$$V = IR$$

input	meaning	unit
`I`	current	`A`
`R`	resistance	`ohm`

- **Output unit:** `V`
- **File:** `ohm__V.onnx`

```
inputs {'I': 2.0, 'R': 2.1}
-> 4.2 V   (ran on ohm__V.onnx; matches the Python engine)
```

Power from current and resistance — solve for current — `ohm_power_current.I`

$$P = I^2R$$

input	meaning	unit
`P`	power dissipated	`W`
`R`	resistance	`ohm`

- **Output unit:** `A`
- **File:** `ohm_power_current__I.onnx`
- **Applies when:** resistive

```
inputs {'P': 2.0, 'R': 2.1}
-> 0.9759 A   (ran on ohm_power_current__I.onnx; matches the Python engine)
```

Power from current and resistance — solve for power dissipated — `ohm_power_current.P`

$$P = I^2 R$$

input	meaning	unit
`I`	current	`A`
`R`	resistance	`ohm`

- **Output unit:** `W`
- **File:** `ohm_power_current__P.onnx`
- **Applies when:** resistive

```
inputs {'I': 2.0, 'R': 2.1}
-> 8.4 W   (ran on ohm_power_current__P.onnx; matches the Python engine)
```

Power from current and resistance — solve for resistance — `ohm_power_current.R`

$$P = I^2 R$$

input	meaning	unit
`P`	power dissipated	`W`
`I`	current	`A`

- **Output unit:** `ohm`
- **File:** `ohm_power_current__R.onnx`
- **Applies when:** resistive

```
inputs {'P': 2.0, 'I': 2.1}
-> 0.453515 ohm   (ran on ohm_power_current__R.onnx; matches the Python engine)
```

Power from voltage and resistance — solve for power dissipated — `ohm_power_voltage.P`

$$P = \frac{V^2}{R}$$

input	meaning	unit
`V`	voltage	`V`
`R`	resistance	`ohm`

- **Output unit:** `W`
- **File:** `ohm_power_voltage__P.onnx`
- **Applies when:** resistive

```
inputs {'V': 2.0, 'R': 2.1}
-> 1.90476 W   (ran on ohm_power_voltage__P.onnx; matches the Python engine)
```

Power from voltage and resistance — solve for resistance — `ohm_power_voltage.R`

$$P = \frac{V^2}{R}$$

input	meaning	unit
`P`	power dissipated	`W`
`V`	voltage	`V`

- **Output unit:** `ohm`
- **File:** `ohm_power_voltage__R.onnx`
- **Applies when:** resistive

```
inputs {'P': 2.0, 'V': 2.1}
-> 2.205 ohm   (ran on ohm_power_voltage__R.onnx; matches the Python engine)
```

Power from voltage and resistance — solve for voltage — `ohm_power_voltage.V`

$$P = \frac{V^2}{R}$$

input	meaning	unit
`P`	power dissipated	`W`
`R`	resistance	`ohm`

- **Output unit:** `V`
- **File:** `ohm_power_voltage__V.onnx`
- **Applies when:** resistive

```
inputs {'P': 2.0, 'R': 2.1}
-> 2.04939 V   (ran on ohm_power_voltage__V.onnx; matches the Python engine)
```

Period of a circular orbit — solve for constant — `orbital_period.G`

$$T = 2\pi\sqrt{\frac{r^3}{GM}}$$

input	meaning	unit
`T`	orbital period	`s`
`r`	orbital radius	`m`
`M`	mass of the central body	`kg`

- **Output unit:** `N*m^2/kg^2`
- **File:** `orbital_period__G.onnx`
- **Applies when:** orbit

```
inputs {'T': 2.0, 'r': 2.1, 'M': 2.2}
-> 41.5465 N*m^2/kg^2   (ran on orbital_period__G.onnx; matches the Python engine)
```

Period of a circular orbit — solve for mass of the central body — `orbital_period.M`

$$T = 2\pi\sqrt{\frac{r^3}{GM}}$$

input	meaning	unit
`T`	orbital period	`s`
`r`	orbital radius	`m`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `kg`
- **File:** `orbital_period__M.onnx`
- **Applies when:** orbit

```
inputs {'T': 2.0, 'r': 2.1, 'G': 2.2}
-> 41.5465 kg   (ran on orbital_period__M.onnx; matches the Python engine)
```

Period of a circular orbit — solve for orbital period — `orbital_period.T`

$$T = 2\pi\sqrt{\frac{r^3}{GM}}$$

input	meaning	unit
`r`	orbital radius	`m`
`M`	mass of the central body	`kg`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `s`
- **File:** `orbital_period__T.onnx`
- **Applies when:** orbit

```
inputs {'r': 2.0, 'M': 2.1, 'G': 2.2}
-> 8.26806 s   (ran on orbital_period__T.onnx; matches the Python engine)
```

Period of a circular orbit — solve for orbital radius — `orbital_period.r`

$$T = 2\pi\sqrt{\frac{r^3}{GM}}$$

input	meaning	unit
`T`	orbital period	`s`

input	meaning	unit
`M`	mass of the central body	`kg`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `m`
- **File:** `orbital_period__r.onnx`
- **Applies when:** orbit

```
inputs {'T': 2.0, 'M': 2.1, 'G': 2.2}
-> 0.776451 m   (ran on orbital_period__r.onnx; matches the Python engine)
```

Speed in a circular orbit — solve for constant — `orbital_speed.G`

$$v = \sqrt{\frac{GM}{r}}$$

input	meaning	unit
`v`	orbital speed	`m/s`
`M`	mass of the central body	`kg`
`r`	orbital radius	`m`

- **Output unit:** `N*m^2/kg^2`
- **File:** `orbital_speed__G.onnx`
- **Applies when:** orbit

```
inputs {'v': 2.0, 'M': 2.1, 'r': 2.2}
-> 4.19048 N*m^2/kg^2   (ran on orbital_speed__G.onnx; matches the Python engine)
```

Speed in a circular orbit — solve for mass of the central body — `orbital_speed.M`

$$v = \sqrt{\frac{GM}{r}}$$

input	meaning	unit
`v`	orbital speed	`m/s`
`r`	orbital radius	`m`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `kg`
- **File:** `orbital_speed__M.onnx`
- **Applies when:** orbit

```
inputs {'v': 2.0, 'r': 2.1, 'G': 2.2}
-> 3.81818 kg   (ran on orbital_speed__M.onnx; matches the Python engine)
```

Speed in a circular orbit — solve for orbital radius — `orbital_speed.r`

$$v = \sqrt{\frac{GM}{r}}$$

input	meaning	unit
`v`	orbital speed	`m/s`
`M`	mass of the central body	`kg`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `m`
- **File:** `orbital_speed__r.onnx`
- **Applies when:** orbit

```
inputs {'v': 2.0, 'M': 2.1, 'G': 2.2}
-> 1.155 m   (ran on orbital_speed__r.onnx; matches the Python engine)
```

Speed in a circular orbit — solve for orbital speed — `orbital_speed.v`

$$v = \sqrt{\frac{GM}{r}}$$

input	meaning	unit
`M`	mass of the central body	`kg`
`r`	orbital radius	`m`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `m/s`
- **File:** `orbital_speed__v.onnx`
- **Applies when:** orbit

```
inputs {'M': 2.0, 'r': 2.1, 'G': 2.2}
-> 1.44749 m/s   (ran on orbital_speed__v.onnx; matches the Python engine)
```

Capacitance of parallel plates — solve for area of each plate — `parallel_plate_capacitor.A`

$$C = \frac{\varepsilon_0 A}{d}$$

input	meaning	unit
`C`	capacitance	`F`
`d`	plate separation	`m`
`eps0`	permittivity	`C^2/(N*m^2)`

- **Output unit:** `m^2`

- **File:** `parallel_plate_capacitor__A.onnx`
- **Applies when:** capacitor

```
inputs {'C': 2.0, 'd': 2.1, 'eps0': 2.2}
-> 1.90909 m^2   (ran on parallel_plate_capacitor__A.onnx; matches the Python engine)
```

Capacitance of parallel plates — solve for capacitance — `parallel_plate_capacitor.C`

$$C = \frac{\varepsilon_0 A}{d}$$

input	meaning	unit
`A`	area of each plate	`m^2`
`d`	plate separation	`m`
`eps0`	permittivity	`C^2/(N*m^2)`

- **Output unit:** `F`
- **File:** `parallel_plate_capacitor__C.onnx`
- **Applies when:** capacitor

```
inputs {'A': 2.0, 'd': 2.1, 'eps0': 2.2}
-> 2.09524 F   (ran on parallel_plate_capacitor__C.onnx; matches the Python engine)
```

Capacitance of parallel plates — solve for plate separation — `parallel_plate_capacitor.d`

$$C = \frac{\varepsilon_0 A}{d}$$

input	meaning	unit
`C`	capacitance	`F`
`A`	area of each plate	`m^2`
`eps0`	permittivity	`C^2/(N*m^2)`

- **Output unit:** `m`
- **File:** `parallel_plate_capacitor__d.onnx`
- **Applies when:** capacitor

```
inputs {'C': 2.0, 'A': 2.1, 'eps0': 2.2}
-> 2.31 m   (ran on parallel_plate_capacitor__d.onnx; matches the Python engine)
```

Capacitance of parallel plates — solve for constant — `parallel_plate_capacitor.eps0`

$$C = \frac{\varepsilon_0 A}{d}$$

input	meaning	unit
`C`	capacitance	`F`
`A`	area of each plate	`m^2`
`d`	plate separation	`m`

- **Output unit:** `C^2/(N*m^2)`
- **File:** `parallel_plate_capacitor__eps0.onnx`
- **Applies when:** capacitor

```
inputs {'C': 2.0, 'A': 2.1, 'd': 2.2}
-> 2.09524 C^2/(N*m^2)   (ran on parallel_plate_capacitor__eps0.onnx; matches the Python engine)
```

Period of a simple pendulum — solve for length of the pendulum — `pendulum_period.Lp`

$$T = 2\pi\sqrt{\frac{L_p}{g}}$$

input	meaning	unit
`T`	period of the swing	`s`
`g`	acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `pendulum_period__Lp.onnx`
- **Applies when:** pendulum

```
inputs {'T': 2.0, 'g': 2.1}
-> 0.212774 m   (ran on pendulum_period__Lp.onnx; matches the Python engine)
```

Period of a simple pendulum — solve for period of the swing — `pendulum_period.T`

$$T = 2\pi\sqrt{\frac{L_p}{g}}$$

input	meaning	unit
`Lp`	length of the pendulum	`m`
`g`	acceleration	`m/s^2`

- **Output unit:** `s`
- **File:** `pendulum_period__T.onnx`
- **Applies when:** pendulum

```
inputs {'Lp': 2.0, 'g': 2.1}
-> 6.13176 s   (ran on pendulum_period__T.onnx; matches the Python engine)
```

Period of a simple pendulum — solve for gravity — `pendulum_period.g`

$$T = 2\pi\sqrt{\frac{L_p}{g}}$$

input	meaning	unit
`T`	period of the swing	`s`
`Lp`	length of the pendulum	`m`

- **Output unit:** `m/s^2`
- **File:** `pendulum_period__g.onnx`
- **Applies when:** pendulum

```
inputs {'T': 2.0, 'Lp': 2.1}
-> 20.7262 m/s^2   (ran on pendulum_period__g.onnx; matches the Python engine)
```

Period and frequency — solve for period — `period_frequency.T`

$$T = \frac{1}{f}$$

input	meaning	unit
`f`	frequency	`Hz`

- **Output unit:** `s`
- **File:** `period_frequency__T.onnx`

```
inputs {'f': 2.0}
-> 0.5 s   (ran on period_frequency__T.onnx; matches the Python engine)
```

Period and frequency — solve for frequency — `period_frequency.f`

$$T = \frac{1}{f}$$

input	meaning	unit
`T`	period	`s`

- **Output unit:** `Hz`
- **File:** `period_frequency__f.onnx`

```
inputs {'T': 2.0}
-> 0.5 Hz   (ran on period_frequency__f.onnx; matches the Python engine)
```

Energy of a photon — solve for photon energy — `photon_energy.E`

$$E = hf$$

input	meaning	unit
`f`	frequency of the light	`Hz`
`h`	planck constant	`J*s`

- **Output unit:** `J`
- **File:** `photon_energy__E.onnx`
- **Applies when:** photon

```
inputs {'f': 2.0, 'h': 2.1}
-> 4.2 J   (ran on photon_energy__E.onnx; matches the Python engine)
```

Energy of a photon — solve for frequency of the light — `photon_energy.f`

$$E = hf$$

input	meaning	unit
`E`	photon energy	`J`
`h`	planck constant	`J*s`

- **Output unit:** `Hz`
- **File:** `photon_energy__f.onnx`
- **Applies when:** photon

```
inputs {'E': 2.0, 'h': 2.1}
-> 0.952381 Hz   (ran on photon_energy__f.onnx; matches the Python engine)
```

Energy of a photon — solve for constant — `photon_energy.h`

$$E = hf$$

input	meaning	unit
`E`	photon energy	`J`
`f`	frequency of the light	`Hz`

- **Output unit:** `J*s`
- **File:** `photon_energy__h.onnx`
- **Applies when:** photon

```
inputs {'E': 2.0, 'f': 2.1}
-> 0.952381 J*s   (ran on photon_energy__h.onnx; matches the Python engine)
```

Photon energy from wavelength — solve for photon — `photon_energy_wavelength.E`

$$E = \frac{hc}{\lambda}$$

input	meaning	unit
`lam`	wavelength in vacuum	`m`
`h`	planck constant	`J*s`
`c`	speed	`m/s`

- **Output unit:** `J`
- **File:** `photon_energy_wavelength__E.onnx`
- **Applies when:** photon

```
inputs {'lam': 2.0, 'h': 2.1, 'c': 2.2}
-> 2.31 J   (ran on photon_energy_wavelength__E.onnx; matches the Python engine)
```

Photon energy from wavelength — solve for light — `photon_energy_wavelength.c`

$$E = \frac{hc}{\lambda}$$

input	meaning	unit
`E`	energy	`J`
`lam`	wavelength in vacuum	`m`
`h`	planck constant	`J*s`

- **Output unit:** `m/s`
- **File:** `photon_energy_wavelength__c.onnx`
- **Applies when:** photon

```
inputs {'E': 2.0, 'lam': 2.1, 'h': 2.2}
-> 1.90909 m/s   (ran on photon_energy_wavelength__c.onnx; matches the Python engine)
```

Photon energy from wavelength — solve for constant — `photon_energy_wavelength.h`

$$E = \frac{hc}{\lambda}$$

input	meaning	unit
`E`	energy	`J`
`lam`	wavelength in vacuum	`m`
`c`	speed	`m/s`

- **Output unit:** `J*s`
- **File:** `photon_energy_wavelength__h.onnx`

- **Applies when:** photon

```
inputs {'E': 2.0, 'lam': 2.1, 'c': 2.2}
-> 1.90909 J*s   (ran on photon_energy_wavelength__h.onnx; matches the Python engine)
```

Photon energy from wavelength — solve for wavelength in vacuum — `photon_energy_wavelength.lam`

$$E = \frac{hc}{\lambda}$$

input	meaning	unit
`E`	energy	`J`
`h`	planck constant	`J*s`
`c`	speed	`m/s`

- **Output unit:** `m`
- **File:** `photon_energy_wavelength__lam.onnx`
- **Applies when:** photon

```
inputs {'E': 2.0, 'h': 2.1, 'c': 2.2}
-> 2.31 m   (ran on photon_energy_wavelength__lam.onnx; matches the Python engine)
```

Momentum of a photon — solve for constant — `photon_momentum.h`

$$p = \frac{h}{\lambda}$$

input	meaning	unit
`p`	photon momentum	`kg*m/s`
`lam`	length	`m`

- **Output unit:** `J*s`
- **File:** `photon_momentum__h.onnx`
- **Applies when:** photon

```
inputs {'p': 2.0, 'lam': 2.1}
-> 4.2 J*s   (ran on photon_momentum__h.onnx; matches the Python engine)
```

Momentum of a photon — solve for wavelength — `photon_momentum.lam`

$$p = \frac{h}{\lambda}$$

input	meaning	unit
`p`	photon momentum	`kg*m/s`

input	meaning	unit
`h`	planck constant	`J*s`

- **Output unit:** `m`
- **File:** `photon_momentum__lam.onnx`
- **Applies when:** photon

```
inputs {'p': 2.0, 'h': 2.1}
-> 1.05 m   (ran on photon_momentum__lam.onnx; matches the Python engine)
```

Momentum of a photon — solve for photon momentum — `photon_momentum.p`

$$p = \frac{h}{\lambda}$$

input	meaning	unit
`lam`	length	`m`
`h`	planck constant	`J*s`

- **Output unit:** `kg*m/s`
- **File:** `photon_momentum__p.onnx`
- **Applies when:** photon

```
inputs {'lam': 2.0, 'h': 2.1}
-> 1.05 kg*m/s   (ran on photon_momentum__p.onnx; matches the Python engine)
```

Electric field of a point charge — solve for field strength at distance r — `point_charge_field.Ef`

$$E_f = \frac{k_e Q}{r^2}$$

input	meaning	unit
`Q`	magnitude of the source charge	`C`
`r`	distance from the charge	`m`
`ke`	coulomb constant	`N*m^2/C^2`

- **Output unit:** `N/C`
- **File:** `point_charge_field__Ef.onnx`
- **Applies when:** electric

```
inputs {'Q': 2.0, 'r': 2.1, 'ke': 2.2}
-> 0.997732 N/C   (ran on point_charge_field__Ef.onnx; matches the Python engine)
```

Electric field of a point charge — solve for magnitude of the source charge — `point_charge_field.Q`

$$E_f = \frac{k_e Q}{r^2}$$

input	meaning	unit
`Ef`	field strength at distance r	`N/C`
`r`	distance from the charge	`m`
`ke`	coulomb constant	`N*m^2/C^2`

- **Output unit:** `C`
- **File:** `point_charge_field__Q.onnx`
- **Applies when:** electric

```
inputs {'Ef': 2.0, 'r': 2.1, 'ke': 2.2}
-> 4.00909 C   (ran on point_charge_field__Q.onnx; matches the Python engine)
```

Electric field of a point charge — solve for constant — `point_charge_field.ke`

$$E_f = \frac{k_e Q}{r^2}$$

input	meaning	unit
`Ef`	field strength at distance r	`N/C`
`Q`	magnitude of the source charge	`C`
`r`	distance from the charge	`m`

- **Output unit:** `N*m^2/C^2`
- **File:** `point_charge_field__ke.onnx`
- **Applies when:** electric

```
inputs {'Ef': 2.0, 'Q': 2.1, 'r': 2.2}
-> 4.60952 N*m^2/C^2   (ran on point_charge_field__ke.onnx; matches the Python engine)
```

Electric field of a point charge — solve for distance from the charge — `point_charge_field.r`

$$E_f = \frac{k_e Q}{r^2}$$

input	meaning	unit
`Ef`	field strength at distance r	`N/C`
`Q`	magnitude of the source charge	`C`
`ke`	coulomb constant	`N*m^2/C^2`

- **Output unit:** `m`
- **File:** `point_charge_field__r.onnx`
- **Applies when:** electric

```
inputs {'Ef': 2.0, 'Q': 2.1, 'ke': 2.2}
-> 1.51987 m   (ran on point_charge_field__r.onnx; matches the Python engine)
```

Electric potential of a point charge — solve for source charge — `point_charge_potential.Q`

$$V = \frac{k_e Q}{r}$$

input	meaning	unit
`V`	electric potential relative to infinity	`V`
`r`	distance from the charge	`m`
`ke`	coulomb constant	`N*m^2/C^2`

- **Output unit:** `C`
- **File:** `point_charge_potential__Q.onnx`
- **Applies when:** electric

```
inputs {'V': 2.0, 'r': 2.1, 'ke': 2.2}
-> 1.90909 C   (ran on point_charge_potential__Q.onnx; matches the Python engine)
```

Electric potential of a point charge — solve for electric potential relative to infinity — `point_charge_potential.V`

$$V = \frac{k_e Q}{r}$$

input	meaning	unit
`Q`	source charge	`C`
`r`	distance from the charge	`m`
`ke`	coulomb constant	`N*m^2/C^2`

- **Output unit:** `V`
- **File:** `point_charge_potential__V.onnx`
- **Applies when:** electric

```
inputs {'Q': 2.0, 'r': 2.1, 'ke': 2.2}
-> 2.09524 V   (ran on point_charge_potential__V.onnx; matches the Python engine)
```

Electric potential of a point charge — solve for constant — `point_charge_potential.ke`

$$V = \frac{k_e Q}{r}$$

input	meaning	unit
`V`	electric potential relative to infinity	`V`
`Q`	source charge	`C`
`r`	distance from the charge	`m`

- **Output unit:** `N*m^2/C^2`
- **File:** `point_charge_potential__ke.onnx`
- **Applies when:** electric

```
inputs {'V': 2.0, 'Q': 2.1, 'r': 2.2}
-> 2.09524 N*m^2/C^2   (ran on point_charge_potential__ke.onnx; matches the Python engine)
```

Electric potential of a point charge — solve for distance from the charge — `point_charge_potential.r`

$$V = \frac{k_e Q}{r}$$

input	meaning	unit
`V`	electric potential relative to infinity	`V`
`Q`	source charge	`C`
`ke`	coulomb constant	`N*m^2/C^2`

- **Output unit:** `m`
- **File:** `point_charge_potential__r.onnx`
- **Applies when:** electric

```
inputs {'V': 2.0, 'Q': 2.1, 'ke': 2.2}
-> 2.31 m   (ran on point_charge_potential__r.onnx; matches the Python engine)
```

Average power — solve for energy transferred — `power_energy.E`

$$P = \frac{E}{t}$$

input	meaning	unit
`P`	average power	`W`
`t`	elapsed time	`s`

- **Output unit:** `J`
- **File:** `power_energy__E.onnx`

```
inputs {'P': 2.0, 't': 2.1}
-> 4.2 J   (ran on power_energy__E.onnx; matches the Python engine)
```

Average power — solve for average power — `power_energy.P`

$$P = \frac{E}{t}$$

input	meaning	unit
`E`	energy transferred	`J`
`t`	elapsed time	`s`

- **Output unit:** `W`
- **File:** `power_energy__P.onnx`

```
inputs {'E': 2.0, 't': 2.1}
-> 0.952381 W   (ran on power_energy__P.onnx; matches the Python engine)
```

Average power — solve for elapsed time — `power_energy.t`

$$P = \frac{E}{t}$$

input	meaning	unit
`P`	average power	`W`
`E`	energy transferred	`J`

- **Output unit:** `s`
- **File:** `power_energy__t.onnx`

```
inputs {'P': 2.0, 'E': 2.1}
-> 1.05 s   (ran on power_energy__t.onnx; matches the Python engine)
```

Power of a force on a moving body — solve for force along the motion — `power_force_speed.F`

$$P = Fv$$

input	meaning	unit
`P`	power	`W`
`v`	speed	`m/s`

- **Output unit:** `N`
- **File:** `power_force_speed__F.onnx`

```
inputs {'P': 2.0, 'v': 2.1}
-> 0.952381 N   (ran on power_force_speed__F.onnx; matches the Python engine)
```

Power of a force on a moving body — solve for power — `power_force_speed.P`

$$P = Fv$$

input	meaning	unit
`F`	force along the motion	`N`
`v`	speed	`m/s`

- **Output unit:** `W`
- **File:** `power_force_speed__P.onnx`

```
inputs {'F': 2.0, 'v': 2.1}
-> 4.2 W   (ran on power_force_speed__P.onnx; matches the Python engine)
```

Power of a force on a moving body — solve for speed — `power_force_speed.v`

$$P = Fv$$

input	meaning	unit
`P`	power	`W`
`F`	force along the motion	`N`

- **Output unit:** `m/s`
- **File:** `power_force_speed__v.onnx`

```
inputs {'P': 2.0, 'F': 2.1}
-> 0.952381 m/s   (ran on power_force_speed__v.onnx; matches the Python engine)
```

Pressure of a force on an area — solve for area — `pressure_force.A`

$$P = \frac{F}{A}$$

input	meaning	unit
`P`	pressure	`Pa`
`F`	force normal to the surface	`N`

- **Output unit:** `m^2`
- **File:** `pressure_force__A.onnx`

```
inputs {'P': 2.0, 'F': 2.1}
-> 1.05 m^2   (ran on pressure_force__A.onnx; matches the Python engine)
```

Pressure of a force on an area — solve for force normal to the surface — `pressure_force.F`

$$P = \frac{F}{A}$$

input	meaning	unit
`P`	pressure	`Pa`
`A`	area	`m^2`

- **Output unit:** `N`
- **File:** `pressure_force__F.onnx`

```
inputs {'P': 2.0, 'A': 2.1}
-> 4.2 N   (ran on pressure_force__F.onnx; matches the Python engine)
```

Pressure of a force on an area — solve for pressure — `pressure_force.P`

$$P = \frac{F}{A}$$

input	meaning	unit
`F`	force normal to the surface	`N`
`A`	area	`m^2`

- **Output unit:** `Pa`
- **File:** `pressure_force__P.onnx`

```
inputs {'F': 2.0, 'A': 2.1}
-> 0.952381 Pa   (ran on pressure_force__P.onnx; matches the Python engine)
```

Voltage of a charging capacitor — solve for capacitance — `rc_charging.C`

$$V = \mathrm{emf}\left(1 - e^{\frac{-t}{RC}}\right)$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`emf`	emf of the source	`V`
`t`	time since charging began	`s`
`R`	resistance	`ohm`

- **Output unit:** `F`
- **File:** `rc_charging__C.onnx`
- **Applies when:** rc_circuit; charging

```
inputs {'V': 2.0, 'emf': 2.1, 't': 2.2, 'R': 2.3}
-> 0.314178 F   (ran on rc_charging__C.onnx; matches the Python engine)
```

Voltage of a charging capacitor — solve for resistance — `rc_charging.R`

$$V = \text{emf}\left(1 - e^{\frac{-t}{RC}}\right)$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`emf`	emf of the source	`V`
`t`	time since charging began	`s`
`C`	capacitance	`F`

- **Output unit:** `ohm`
- **File:** `rc_charging__R.onnx`
- **Applies when:** rc_circuit; charging

```
inputs {'V': 2.0, 'emf': 2.1, 't': 2.2, 'C': 2.3}
-> 0.314178 ohm   (ran on rc_charging__R.onnx; matches the Python engine)
```

Voltage of a charging capacitor — solve for voltage across the capacitor at time t — `rc_charging.V`

$$V = \text{emf}\left(1 - e^{\frac{-t}{RC}}\right)$$

input	meaning	unit
`emf`	emf of the source	`V`
`t`	time since charging began	`s`
`R`	resistance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `V`
- **File:** `rc_charging__V.onnx`
- **Applies when:** rc_circuit; charging

```
inputs {'emf': 2.0, 't': 2.1, 'R': 2.2, 'C': 2.3}
-> 0.679346 V   (ran on rc_charging__V.onnx; matches the Python engine)
```

Voltage of a charging capacitor — solve for emf of the source — `rc_charging.emf`

$$V = \text{emf}\left(1 - e^{\frac{-t}{RC}}\right)$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`t`	time since charging began	`s`
`R`	resistance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `V`
- **File:** `rc_charging__emf.onnx`
- **Applies when:** rc_circuit; charging

```
inputs {'V': 2.0, 't': 2.1, 'R': 2.2, 'C': 2.3}
-> 5.88802 V   (ran on rc_charging__emf.onnx; matches the Python engine)
```

Voltage of a charging capacitor — solve for time since charging began — `rc_charging.t`

$$V = \text{emf}\left(1 - e^{\frac{-t}{RC}}\right)$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`emf`	emf of the source	`V`
`R`	resistance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `s`
- **File:** `rc_charging__t.onnx`
- **Applies when:** rc_circuit; charging

```
inputs {'V': 2.0, 'emf': 2.1, 'R': 2.2, 'C': 2.3}
-> 15.4053 s   (ran on rc_charging__t.onnx; matches the Python engine)
```

Voltage of a discharging capacitor — solve for capacitance — `rc_discharging.C`

$$V = V_0 e^{\frac{-t}{RC}}$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`V0`	initial voltage across the capacitor	`V`
`t`	time since the discharge began	`s`
`R`	resistance	`ohm`

- **Output unit:** `F`
- **File:** `rc_discharging__C.onnx`
- **Applies when:** rc_circuit; discharging

```
inputs {'V': 2.0, 'V0': 2.1, 't': 2.2, 'R': 2.3}
-> 19.6048 F   (ran on rc_discharging__C.onnx; matches the Python engine)
```

Voltage of a discharging capacitor — solve for resistance — `rc_discharging.R`

$$V = V_0 e^{\frac{-t}{RC}}$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`V0`	initial voltage across the capacitor	`V`
`t`	time since the discharge began	`s`
`C`	capacitance	`F`

- **Output unit:** `ohm`
- **File:** `rc_discharging__R.onnx`
- **Applies when:** rc_circuit; discharging

```
inputs {'V': 2.0, 'V0': 2.1, 't': 2.2, 'C': 2.3}
-> 19.6048 ohm   (ran on rc_discharging__R.onnx; matches the Python engine)
```

Voltage of a discharging capacitor — solve for voltage across the capacitor at time t — `rc_discharging.V`

$$V = V_0 e^{\frac{-t}{RC}}$$

input	meaning	unit
`V0`	initial voltage across the capacitor	`V`
`t`	time since the discharge began	`s`
`R`	resistance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `V`
- **File:** `rc_discharging__V.onnx`
- **Applies when:** rc_circuit; discharging

```
inputs {'V0': 2.0, 't': 2.1, 'R': 2.2, 'C': 2.3}
-> 1.32065 V   (ran on rc_discharging__V.onnx; matches the Python engine)
```

Voltage of a discharging capacitor — solve for initial voltage across the capacitor — `rc_discharging.V0`

$$V = V_0 e^{\frac{-t}{RC}}$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`t`	time since the discharge began	`s`
`R`	resistance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `V`
- **File:** `rc_discharging__V0.onnx`
- **Applies when:** rc_circuit; discharging

```
inputs {'V': 2.0, 't': 2.1, 'R': 2.2, 'C': 2.3}
-> 3.0288 V   (ran on rc_discharging__V0.onnx; matches the Python engine)
```

Voltage of a discharging capacitor — solve for time since the discharge began — `rc_discharging.t`

$$V = V_0 e^{\frac{-t}{RC}}$$

input	meaning	unit
`V`	voltage across the capacitor at time t	`V`
`V0`	initial voltage across the capacitor	`V`
`R`	resistance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `s`
- **File:** `rc_discharging__t.onnx`
- **Applies when:** rc_circuit; discharging

```
inputs {'V': 2.0, 'V0': 2.1, 'R': 2.2, 'C': 2.3}
-> 0.246878 s   (ran on rc_discharging__t.onnx; matches the Python engine)
```

Time constant of an RC circuit — solve for capacitance — `rc_time_constant.C`

$$\tau = RC$$

input	meaning	unit
`tau`	time constant	`s`
`R`	resistance	`ohm`

- **Output unit:** `F`
- **File:** `rc_time_constant__C.onnx`
- **Applies when:** rc_circuit

```
inputs {'tau': 2.0, 'R': 2.1}
-> 0.952381 F   (ran on rc_time_constant__C.onnx; matches the Python engine)
```

Time constant of an RC circuit — solve for resistance — `rc_time_constant.R`

$$\tau = RC$$

input	meaning	unit
`tau`	time constant	`s`
`C`	capacitance	`F`

- **Output unit:** `ohm`
- **File:** `rc_time_constant__R.onnx`
- **Applies when:** rc_circuit

```
inputs {'tau': 2.0, 'C': 2.1}
-> 0.952381 ohm   (ran on rc_time_constant__R.onnx; matches the Python engine)
```

Time constant of an RC circuit — solve for time constant — `rc_time_constant.tau`

$$\tau = RC$$

input	meaning	unit
`R`	resistance	`ohm`
`C`	capacitance	`F`

- **Output unit:** `s`
- **File:** `rc_time_constant__tau.onnx`
- **Applies when:** rc_circuit

```
inputs {'R': 2.0, 'C': 2.1}
-> 4.2 s   (ran on rc_time_constant__tau.onnx; matches the Python engine)
```

Resistance of a uniform wire — solve for cross-sectional area — `resistivity.A`

$$R = \frac{\rho_e L}{A}$$

input	meaning	unit
`R`	resistance	`ohm`
`rho_e`	resistivity of the material	`ohm*m`
`L`	length of the wire	`m`

- **Output unit:** `m^2`
- **File:** `resistivity__A.onnx`

- **Applies when:** wire

```
inputs {'R': 2.0, 'rho_e': 2.1, 'L': 2.2}
-> 2.31 m^2   (ran on resistivity__A.onnx; matches the Python engine)
```

Resistance of a uniform wire — solve for length of the wire — `resistivity.L`

$$R = \frac{\rho_e L}{A}$$

input	meaning	unit
`R`	resistance	`ohm`
`rho_e`	resistivity of the material	`ohm*m`
`A`	cross-sectional area	`m^2`

- **Output unit:** `m`
- **File:** `resistivity__L.onnx`
- **Applies when:** wire

```
inputs {'R': 2.0, 'rho_e': 2.1, 'A': 2.2}
-> 2.09524 m   (ran on resistivity__L.onnx; matches the Python engine)
```

Resistance of a uniform wire — solve for resistance — `resistivity.R`

$$R = \frac{\rho_e L}{A}$$

input	meaning	unit
`rho_e`	resistivity of the material	`ohm*m`
`L`	length of the wire	`m`
`A`	cross-sectional area	`m^2`

- **Output unit:** `ohm`
- **File:** `resistivity__R.onnx`
- **Applies when:** wire

```
inputs {'rho_e': 2.0, 'L': 2.1, 'A': 2.2}
-> 1.90909 ohm   (ran on resistivity__R.onnx; matches the Python engine)
```

Resistance of a uniform wire — solve for resistivity of the material — `resistivity.rho_e`

$$R = \frac{\rho_e L}{A}$$

input	meaning	unit
`R`	resistance	`ohm`

input	meaning	unit
`L`	length of the wire	`m`
`A`	cross-sectional area	`m^2`

- **Output unit:** `ohm*m`
- **File:** `resistivity__rho_e.onnx`
- **Applies when:** wire

```
inputs {'R': 2.0, 'L': 2.1, 'A': 2.2}
-> 2.09524 ohm*m   (ran on resistivity__rho_e.onnx; matches the Python engine)
```

Rest energy — solve for rest energy — `rest_energy.E0`

$$E_0 = mc^2$$

input	meaning	unit
`m`	rest mass	`kg`
`c`	speed	`m/s`

- **Output unit:** `J`
- **File:** `rest_energy__E0.onnx`
- **Applies when:** mass_energy

```
inputs {'m': 2.0, 'c': 2.1}
-> 8.82 J   (ran on rest_energy__E0.onnx; matches the Python engine)
```

Rest energy — solve for light — `rest_energy.c`

$$E_0 = mc^2$$

input	meaning	unit
`E0`	rest energy	`J`
`m`	rest mass	`kg`

- **Output unit:** `m/s`
- **File:** `rest_energy__c.onnx`
- **Applies when:** mass_energy

```
inputs {'E0': 2.0, 'm': 2.1}
-> 0.9759 m/s   (ran on rest_energy__c.onnx; matches the Python engine)
```

Rest energy — solve for rest mass — `rest_energy.m`

$$E_0 = mc^2$$

input	meaning	unit
`E0`	rest energy	`J`
`c`	speed	`m/s`

- **Output unit:** `kg`
- **File:** `rest_energy__m.onnx`
- **Applies when:** mass_energy

```
inputs {'E0': 2.0, 'c': 2.1}
-> 0.453515 kg   (ran on rest_energy__m.onnx; matches the Python engine)
```

Current dying away in an RL circuit — solve for current at time t — `rl_current_decay.I`

$$I = I_0 e^{\frac{-t}{\tau}}$$

input	meaning	unit
`I0`	initial current	`A`
`t`	time since the circuit was switched off	`s`
`tau`	time constant	`s`

- **Output unit:** `A`
- **File:** `rl_current_decay__I.onnx`
- **Applies when:** inductor; current_decay

```
inputs {'I0': 2.0, 't': 2.1, 'tau': 2.2}
-> 0.769974 A   (ran on rl_current_decay__I.onnx; matches the Python engine)
```

Current dying away in an RL circuit — solve for initial current — `rl_current_decay.I0`

$$I = I_0 e^{\frac{-t}{\tau}}$$

input	meaning	unit
`I`	current at time t	`A`
`t`	time since the circuit was switched off	`s`
`tau`	time constant	`s`

- **Output unit:** `A`
- **File:** `rl_current_decay__I0.onnx`
- **Applies when:** inductor; current_decay

```
inputs {'I': 2.0, 't': 2.1, 'tau': 2.2}
-> 5.19498 A   (ran on rl_current_decay__I0.onnx; matches the Python engine)
```

Current dying away in an RL circuit — solve for time since the circuit was switched off — `rl_current_decay.t`

$$I = I_0 e^{\frac{-t}{\tau}}$$

input	meaning	unit
`I`	current at time t	`A`
`I0`	initial current	`A`
`tau`	time constant	`s`

- **Output unit:** `s`
- **File:** `rl_current_decay__t.onnx`
- **Applies when:** inductor; current_decay

```
inputs {'I': 2.0, 'I0': 2.1, 'tau': 2.2}
-> 0.107338 s   (ran on rl_current_decay__t.onnx; matches the Python engine)
```

Current dying away in an RL circuit — solve for time constant — `rl_current_decay.tau`

$$I = I_0 e^{\frac{-t}{\tau}}$$

input	meaning	unit
`I`	current at time t	`A`
`I0`	initial current	`A`
`t`	time since the circuit was switched off	`s`

- **Output unit:** `s`
- **File:** `rl_current_decay__tau.onnx`
- **Applies when:** inductor; current_decay

```
inputs {'I': 2.0, 'I0': 2.1, 't': 2.2}
-> 45.0911 s   (ran on rl_current_decay__tau.onnx; matches the Python engine)
```

Time constant of an RL circuit — solve for inductance — `rl_time_constant.L`

$$\tau = \frac{L}{R}$$

input	meaning	unit
`tau`	time constant	`s`
`R`	resistance	`ohm`

- **Output unit:** `H`
- **File:** `rl_time_constant__L.onnx`
- **Applies when:** inductor

```
inputs {'tau': 2.0, 'R': 2.1}
-> 4.2 H   (ran on rl_time_constant__L.onnx; matches the Python engine)
```

Time constant of an RL circuit — solve for resistance — `rl_time_constant.R`

$$\tau = \frac{L}{R}$$

input	meaning	unit
`tau`	time constant	`s`
`L`	inductance	`H`

- **Output unit:** `ohm`
- **File:** `rl_time_constant__R.onnx`
- **Applies when:** inductor

```
inputs {'tau': 2.0, 'L': 2.1}
-> 1.05 ohm   (ran on rl_time_constant__R.onnx; matches the Python engine)
```

Time constant of an RL circuit — solve for time constant — `rl_time_constant.tau`

$$\tau = \frac{L}{R}$$

input	meaning	unit
`L`	inductance	`H`
`R`	resistance	`ohm`

- **Output unit:** `s`
- **File:** `rl_time_constant__tau.onnx`
- **Applies when:** inductor

```
inputs {'L': 2.0, 'R': 2.1}
-> 0.952381 s   (ran on rl_time_constant__tau.onnx; matches the Python engine)
```

Rms current of a sinusoidal supply — solve for peak current — `rms_current.I0`

$$I_{\mathrm{rms}} = \frac{I_0}{\sqrt{2}}$$

input	meaning	unit
`Irms`	rms current	`A`

- **Output unit:** `A`
- **File:** `rms_current__I0.onnx`
- **Applies when:** alternating_current

```
inputs {'Irms': 2.0}
-> 2.82843 A   (ran on rms_current__I0.onnx; matches the Python engine)
```

Rms current of a sinusoidal supply — solve for rms current — `rms_current.Irms`

$$I_{\text{rms}} = \frac{I_0}{\sqrt{2}}$$

input	meaning	unit
`I0`	peak current	`A`

- **Output unit:** `A`
- **File:** `rms_current__Irms.onnx`
- **Applies when:** alternating_current

```
inputs {'I0': 2.0}
-> 1.41421 A   (ran on rms_current__Irms.onnx; matches the Python engine)
```

Rms voltage of a sinusoidal supply — solve for peak voltage — `rms_voltage.V0`

$$V_{\text{rms}} = \frac{V_0}{\sqrt{2}}$$

input	meaning	unit
`Vrms`	rms voltage	`V`

- **Output unit:** `V`
- **File:** `rms_voltage__V0.onnx`
- **Applies when:** alternating_current

```
inputs {'Vrms': 2.0}
-> 2.82843 V   (ran on rms_voltage__V0.onnx; matches the Python engine)
```

Rms voltage of a sinusoidal supply — solve for rms voltage — `rms_voltage.Vrms`

$$V_{\text{rms}} = \frac{V_0}{\sqrt{2}}$$

input	meaning	unit
`V0`	peak voltage	`V`

- **Output unit:** `V`
- **File:** `rms_voltage__Vrms.onnx`
- **Applies when:** alternating_current

```
inputs {'V0': 2.0}
-> 1.41421 V   (ran on rms_voltage__Vrms.onnx; matches the Python engine)
```

Rotational kinetic energy — solve for moment of inertia — `rotational_kinetic_energy.Ir`

$$K_R = \frac{I_r \omega^2}{2}$$

input	meaning	unit
`KR`	rotational kinetic energy	`J`
`omega`	angular velocity	`rad/s`

- **Output unit:** `kg*m^2`
- **File:** `rotational_kinetic_energy__Ir.onnx`
- **Applies when:** rotating

```
inputs {'KR': 2.0, 'omega': 2.1}
-> 0.907029 kg*m^2   (ran on rotational_kinetic_energy__Ir.onnx; matches the Python engine)
```

Rotational kinetic energy — solve for rotational kinetic energy — `rotational_kinetic_energy.KR`

$$K_R = \frac{I_r \omega^2}{2}$$

input	meaning	unit
`Ir`	moment of inertia	`kg*m^2`
`omega`	angular velocity	`rad/s`

- **Output unit:** `J`
- **File:** `rotational_kinetic_energy__KR.onnx`
- **Applies when:** rotating

```
inputs {'Ir': 2.0, 'omega': 2.1}
-> 4.41 J   (ran on rotational_kinetic_energy__KR.onnx; matches the Python engine)
```

Rotational kinetic energy — solve for angular velocity — `rotational_kinetic_energy.omega`

$$K_R = \frac{I_r \omega^2}{2}$$

input	meaning	unit
`KR`	rotational kinetic energy	`J`
`Ir`	moment of inertia	`kg*m^2`

- **Output unit:** `rad/s`

- **File:** `rotational_kinetic_energy__omega.onnx`
- **Applies when:** rotating

```
inputs {'KR': 2.0, 'Ir': 2.1}
-> 1.38013 rad/s   (ran on rotational_kinetic_energy__omega.onnx; matches the Python engine)
```

Maximum speed of a mass on a spring — solve for amplitude — `shm_max_speed.X`

$$v = X\sqrt{\frac{k}{m}}$$

input	meaning	unit
`v`	maximum speed (at equilibrium)	`m/s`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`

- **Output unit:** `m`
- **File:** `shm_max_speed__X.onnx`
- **Applies when:** shm

```
inputs {'v': 2.0, 'k': 2.1, 'm': 2.2}
-> 2.04707 m   (ran on shm_max_speed__X.onnx; matches the Python engine)
```

Maximum speed of a mass on a spring — solve for spring constant — `shm_max_speed.k`

$$v = X\sqrt{\frac{k}{m}}$$

input	meaning	unit
`v`	maximum speed (at equilibrium)	`m/s`
`X`	amplitude	`m`
`m`	oscillating mass	`kg`

- **Output unit:** `N/m`
- **File:** `shm_max_speed__k.onnx`
- **Applies when:** shm

```
inputs {'v': 2.0, 'X': 2.1, 'm': 2.2}
-> 1.99546 N/m   (ran on shm_max_speed__k.onnx; matches the Python engine)
```

Maximum speed of a mass on a spring — solve for oscillating mass — `shm_max_speed.m`

$$v = X\sqrt{\frac{k}{m}}$$

input	meaning	unit
`v`	maximum speed (at equilibrium)	`m/s`
`X`	amplitude	`m`
`k`	spring constant	`N/m`

- **Output unit:** `kg`
- **File:** `shm_max_speed__m.onnx`
- **Applies when:** shm

```
inputs {'v': 2.0, 'X': 2.1, 'k': 2.2}
-> 2.4255 kg   (ran on shm_max_speed__m.onnx; matches the Python engine)
```

Maximum speed of a mass on a spring — solve for maximum speed (at equilibrium) — `shm_max_speed.v`

$$v = X\sqrt{\frac{k}{m}}$$

input	meaning	unit
`X`	amplitude	`m`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`

- **Output unit:** `m/s`
- **File:** `shm_max_speed__v.onnx`
- **Applies when:** shm

```
inputs {'X': 2.0, 'k': 2.1, 'm': 2.2}
-> 1.95402 m/s   (ran on shm_max_speed__v.onnx; matches the Python engine)
```

Magnetic field inside a long solenoid — solve for magnetic field — `solenoid_field.B`

$$B = \mu_0 n_t I$$

input	meaning	unit
`nt`	turns per unit length	`1/m`
`I`	current	`A`
`mu0`	permeability	`T*m/A`

- **Output unit:** `T`
- **File:** `solenoid_field__B.onnx`
- **Applies when:** solenoid

```
inputs {'nt': 2.0, 'I': 2.1, 'mu0': 2.2}
-> 9.24 T   (ran on solenoid_field__B.onnx; matches the Python engine)
```

Magnetic field inside a long solenoid — solve for current — `solenoid_field.I`

$$B = \mu_0 n_t I$$

input	meaning	unit
`B`	magnetic field	`T`
`nt`	turns per unit length	`1/m`
`mu0`	permeability	`T*m/A`

- **Output unit:** `A`
- **File:** `solenoid_field__I.onnx`
- **Applies when:** solenoid

```
inputs {'B': 2.0, 'nt': 2.1, 'mu0': 2.2}
-> 0.4329 A   (ran on solenoid_field__I.onnx; matches the Python engine)
```

Magnetic field inside a long solenoid — solve for constant — `solenoid_field.mu0`

$$B = \mu_0 n_t I$$

input	meaning	unit
`B`	magnetic field	`T`
`nt`	turns per unit length	`1/m`
`I`	current	`A`

- **Output unit:** `T*m/A`
- **File:** `solenoid_field__mu0.onnx`
- **Applies when:** solenoid

```
inputs {'B': 2.0, 'nt': 2.1, 'I': 2.2}
-> 0.4329 T*m/A   (ran on solenoid_field__mu0.onnx; matches the Python engine)
```

Magnetic field inside a long solenoid — solve for turns per unit length — `solenoid_field.nt`

$$B = \mu_0 n_t I$$

input	meaning	unit
`B`	magnetic field	`T`
`I`	current	`A`
`mu0`	permeability	`T*m/A`

- **Output unit:** `1/m`
- **File:** `solenoid_field__nt.onnx`
- **Applies when:** solenoid

```
inputs {'B': 2.0, 'I': 2.1, 'mu0': 2.2}
-> 0.4329 1/m   (ran on solenoid_field__nt.onnx; matches the Python engine)
```

Speed after falling from rest — solve for gravitational acceleration — `speed_after_fall_from_rest.g`

$$v = \sqrt{2gh}$$

input	meaning	unit
`v`	speed after the fall	`m/s`
`h`	height fallen	`m`

- **Output unit:** `m/s^2`
- **File:** `speed_after_fall_from_rest__g.onnx`
- **Applies when:** free_fall; from_rest

```
inputs {'v': 2.0, 'h': 2.1}
-> 0.952381 m/s^2   (ran on speed_after_fall_from_rest__g.onnx; matches the Python engine)
```

Speed after falling from rest — solve for height fallen — `speed_after_fall_from_rest.h`

$$v = \sqrt{2gh}$$

input	meaning	unit
`v`	speed after the fall	`m/s`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `speed_after_fall_from_rest__h.onnx`
- **Applies when:** free_fall; from_rest

```
inputs {'v': 2.0, 'g': 2.1}
-> 0.952381 m   (ran on speed_after_fall_from_rest__h.onnx; matches the Python engine)
```

Speed after falling from rest — solve for speed after the fall — `speed_after_fall_from_rest.v`

$$v = \sqrt{2gh}$$

input	meaning	unit
`g`	gravitational acceleration	`m/s^2`
`h`	height fallen	`m`

- **Output unit:** `m/s`
- **File:** `speed_after_fall_from_rest__v.onnx`
- **Applies when:** free_fall; from_rest

```
inputs {'g': 2.0, 'h': 2.1}
-> 2.89828 m/s   (ran on speed_after_fall_from_rest__v.onnx; matches the Python engine)
```

Energy stored in a linear spring — solve for elastic potential energy — `spring_energy.U`

$$U = \frac{kx^2}{2}$$

input	meaning	unit
`k`	spring constant	`N/m`
`x`	extension or compression	`m`

- **Output unit:** `J`
- **File:** `spring_energy__U.onnx`

```
inputs {'k': 2.0, 'x': 2.1}
-> 4.41 J   (ran on spring_energy__U.onnx; matches the Python engine)
```

Energy stored in a linear spring — solve for spring constant — `spring_energy.k`

$$U = \frac{kx^2}{2}$$

input	meaning	unit
`U`	elastic potential energy	`J`
`x`	extension or compression	`m`

- **Output unit:** `N/m`
- **File:** `spring_energy__k.onnx`

```
inputs {'U': 2.0, 'x': 2.1}
-> 0.907029 N/m   (ran on spring_energy__k.onnx; matches the Python engine)
```

Energy stored in a linear spring — solve for extension or compression — `spring_energy.x`

$$U = \frac{kx^2}{2}$$

input	meaning	unit
`U`	elastic potential energy	`J`
`k`	spring constant	`N/m`

- **Output unit:** `m`
- **File:** `spring_energy__x.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'U': 2.0, 'k': 2.1}
-> -1.38013, 1.38013 m   (ran on spring_energy__x.onnx; matches the Python engine)
```

Force of a linear spring — solve for spring force (magnitude) — `spring_force.F`

$$F = kx$$

input	meaning	unit
`k`	spring constant	`N/m`
`x`	extension or compression	`m`

- **Output unit:** `N`
- **File:** `spring_force__F.onnx`

```
inputs {'k': 2.0, 'x': 2.1}
-> 4.2 N   (ran on spring_force__F.onnx; matches the Python engine)
```

Force of a linear spring — solve for spring constant — `spring_force.k`

$$F = kx$$

input	meaning	unit
`F`	spring force (magnitude)	`N`
`x`	extension or compression	`m`

- **Output unit:** `N/m`
- **File:** `spring_force__k.onnx`

```
inputs {'F': 2.0, 'x': 2.1}
-> 0.952381 N/m   (ran on spring_force__k.onnx; matches the Python engine)
```

Force of a linear spring — solve for extension or compression — `spring_force.x`

$$F = kx$$

input	meaning	unit
`F`	spring force (magnitude)	`N`
`k`	spring constant	`N/m`

- **Output unit:** `m`
- **File:** `spring_force__x.onnx`

```
inputs {'F': 2.0, 'k': 2.1}
-> 0.952381 m   (ran on spring_force__x.onnx; matches the Python engine)
```

Speed of a body launched by a spring — solve for spring constant — `spring_launch_speed.k`

$$\frac{kx^2}{2} = \frac{mv^2}{2}$$

input	meaning	unit
`x`	compression of the spring before release	`m`
`m`	mass of the body	`kg`
`v`	speed when the spring has returned to its natural length	`m/s`

- **Output unit:** `N/m`
- **File:** `spring_launch_speed__k.onnx`
- **Applies when:** spring_launch; frictionless

```
inputs {'x': 2.0, 'm': 2.1, 'v': 2.2}
-> 2.541 N/m   (ran on spring_launch_speed__k.onnx; matches the Python engine)
```

Speed of a body launched by a spring — solve for mass of the body — `spring_launch_speed.m`

$$\frac{kx^2}{2} = \frac{mv^2}{2}$$

input	meaning	unit
`k`	spring constant	`N/m`
`x`	compression of the spring before release	`m`
`v`	speed when the spring has returned to its natural length	`m/s`

- **Output unit:** `kg`
- **File:** `spring_launch_speed__m.onnx`
- **Applies when:** spring_launch; frictionless

```
inputs {'k': 2.0, 'x': 2.1, 'v': 2.2}
-> 1.82231 kg   (ran on spring_launch_speed__m.onnx; matches the Python engine)
```

Speed of a body launched by a spring — solve for speed when the spring has returned to its natural length — `spring_launch_speed.v`

$$\frac{kx^2}{2} = \frac{mv^2}{2}$$

input	meaning	unit
`k`	spring constant	`N/m`
`x`	compression of the spring before release	`m`
`m`	mass of the body	`kg`

- **Output unit:** `m/s`
- **File:** `spring_launch_speed__v.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** spring_launch; frictionless

```
inputs {'k': 2.0, 'x': 2.1, 'm': 2.2}
-> -2.00227, 2.00227 m/s   (ran on spring_launch_speed__v.onnx; matches the Python engine)
```

Speed of a body launched by a spring — solve for compression of the spring before release — `spring_launch_speed.x`

$$\frac{kx^2}{2} = \frac{mv^2}{2}$$

input	meaning	unit
`k`	spring constant	`N/m`
`m`	mass of the body	`kg`
`v`	speed when the spring has returned to its natural length	`m/s`

- **Output unit:** `m`
- **File:** `spring_launch_speed__x.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** spring_launch; frictionless

```
inputs {'k': 2.0, 'm': 2.1, 'v': 2.2}
-> -2.25433, 2.25433 m   (ran on spring_launch_speed__x.onnx; matches the Python engine)
```

Period of a mass on a spring — solve for period — `spring_oscillation_period.T`

$$T = 2\pi\sqrt{\frac{m}{k}}$$

input	meaning	unit
`m`	mass	`kg`
`k`	spring constant	`N/m`

- **Output unit:** `s`
- **File:** `spring_oscillation_period__T.onnx`
- **Applies when:** spring_oscillation

```
inputs {'m': 2.0, 'k': 2.1}
-> 6.13176 s   (ran on spring_oscillation_period__T.onnx; matches the Python engine)
```

Period of a mass on a spring — solve for spring constant — `spring_oscillation_period.k`

$$T = 2\pi\sqrt{\frac{m}{k}}$$

input	meaning	unit
`T`	time	`s`
`m`	mass	kg

- **Output unit:** `N/m`
- **File:** `spring_oscillation_period__k.onnx`
- **Applies when:** spring_oscillation

```
inputs {'T': 2.0, 'm': 2.1}
-> 20.7262 N/m   (ran on spring_oscillation_period__k.onnx; matches the Python engine)
```

Period of a mass on a spring — solve for mass — `spring_oscillation_period.m`

$$T = 2\pi\sqrt{\frac{m}{k}}$$

input	meaning	unit
`T`	time	`s`
`k`	spring constant	`N/m`

- **Output unit:** `kg`
- **File:** `spring_oscillation_period__m.onnx`
- **Applies when:** spring_oscillation

```
inputs {'T': 2.0, 'k': 2.1}
-> 0.212774 kg   (ran on spring_oscillation_period__m.onnx; matches the Python engine)
```

Gravitational acceleration at a distance from a mass — solve for constant — `surface_gravity.G`

$$g = \frac{GM}{r^2}$$

input	meaning	unit
`g`	gravitational acceleration	`m/s^2`
`M`	mass of the planet or star	kg
`r`	distance from the centre	`m`

- **Output unit:** `N*m^2/kg^2`
- **File:** `surface_gravity__G_upper.onnx`

- **Applies when:** celestial

```
inputs {'g': 2.0, 'M': 2.1, 'r': 2.2}
-> 4.60952 N*m^2/kg^2   (ran on surface_gravity__G_upper.onnx; matches the Python engine)
```

Gravitational acceleration at a distance from a mass — solve for mass of the planet or star — `surface_gravity.M`

$$g = \frac{GM}{r^2}$$

input	meaning	unit
`g`	gravitational acceleration	`m/s^2`
`r`	distance from the centre	`m`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `kg`
- **File:** `surface_gravity__M.onnx`
- **Applies when:** celestial

```
inputs {'g': 2.0, 'r': 2.1, 'G': 2.2}
-> 4.00909 kg   (ran on surface_gravity__M.onnx; matches the Python engine)
```

Gravitational acceleration at a distance from a mass — solve for gravitational acceleration — `surface_gravity.g`

$$g = \frac{GM}{r^2}$$

input	meaning	unit
`M`	mass of the planet or star	`kg`
`r`	distance from the centre	`m`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `m/s^2`
- **File:** `surface_gravity__g.onnx`
- **Applies when:** celestial

```
inputs {'M': 2.0, 'r': 2.1, 'G': 2.2}
-> 0.997732 m/s^2   (ran on surface_gravity__g.onnx; matches the Python engine)
```

Gravitational acceleration at a distance from a mass — solve for distance from the centre — `surface_gravity.r`

$$g = \frac{GM}{r^2}$$

input	meaning	unit
`g`	gravitational acceleration	`m/s^2`
`M`	mass of the planet or star	`kg`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `m`
- **File:** `surface_gravity__r.onnx`
- **Applies when:** celestial

```
inputs {'g': 2.0, 'M': 2.1, 'G': 2.2}
-> 1.51987 m   (ran on surface_gravity__r.onnx; matches the Python engine)
```

Tangential acceleration of a point on a rotating body — solve for tangential acceleration — `tangential_acceleration.a`

$$a = r\alpha$$

input	meaning	unit
`r`	distance from the axis	`m`
`alpha`	angular acceleration	`rad/s^2`

- **Output unit:** `m/s^2`
- **File:** `tangential_acceleration__a.onnx`
- **Applies when:** rotating

```
inputs {'r': 2.0, 'alpha': 2.1}
-> 4.2 m/s^2   (ran on tangential_acceleration__a.onnx; matches the Python engine)
```

Tangential acceleration of a point on a rotating body — solve for angular acceleration — `tangential_acceleration.alpha`

$$a = r\alpha$$

input	meaning	unit
`a`	tangential acceleration	`m/s^2`
`r`	distance from the axis	`m`

- **Output unit:** `rad/s^2`
- **File:** `tangential_acceleration__alpha.onnx`
- **Applies when:** rotating

```
inputs {'a': 2.0, 'r': 2.1}
-> 0.952381 rad/s^2   (ran on tangential_acceleration__alpha.onnx; matches the Python engine)
```

Tangential acceleration of a point on a rotating body — solve for distance from the axis — `tangential_acceleration.r`

$$a = r\alpha$$

input	meaning	unit
`a`	tangential acceleration	`m/s^2`
`alpha`	angular acceleration	`rad/s^2`

- **Output unit:** `m`
- **File:** `tangential_acceleration__r.onnx`
- **Applies when:** rotating

```
inputs {'a': 2.0, 'alpha': 2.1}
-> 0.952381 m   (ran on tangential_acceleration__r.onnx; matches the Python engine)
```

Speed on a circle from angular speed — solve for angular speed — `tangential_speed.omega`

$$v = \omega r$$

input	meaning	unit
`v`	speed	`m/s`
`r`	radius of the circle	`m`

- **Output unit:** `rad/s`
- **File:** `tangential_speed__omega.onnx`

```
inputs {'v': 2.0, 'r': 2.1}
-> 0.952381 rad/s   (ran on tangential_speed__omega.onnx; matches the Python engine)
```

Speed on a circle from angular speed — solve for radius of the circle — `tangential_speed.r`

$$v = \omega r$$

input	meaning	unit
`v`	speed	`m/s`
`omega`	angular speed	`rad/s`

- **Output unit:** `m`
- **File:** `tangential_speed__r.onnx`

```
inputs {'v': 2.0, 'omega': 2.1}
-> 0.952381 m   (ran on tangential_speed__r.onnx; matches the Python engine)
```

Speed on a circle from angular speed — solve for speed — `tangential_speed.v`

$$v = \omega r$$

input	meaning	unit
`omega`	angular speed	`rad/s`
`r`	radius of the circle	`m`

- **Output unit:** `m/s`
- **File:** `tangential_speed__v.onnx`

```
inputs {'omega': 2.0, 'r': 2.1}
-> 4.2 m/s   (ran on tangential_speed__v.onnx; matches the Python engine)
```

Tension in a rope holding a hanging body — solve for tension in the rope — `tension_hanging_at_rest.T`

$$T = mg$$

input	meaning	unit
`m`	mass of the hanging body	`kg`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `N`
- **File:** `tension_hanging_at_rest__T.onnx`
- **Applies when:** hanging

```
inputs {'m': 2.0, 'g': 2.1}
-> 4.2 N   (ran on tension_hanging_at_rest__T.onnx; matches the Python engine)
```

Tension in a rope holding a hanging body — solve for gravitational acceleration — `tension_hanging_at_rest.g`

$$T = mg$$

input	meaning	unit
`T`	tension in the rope	`N`
`m`	mass of the hanging body	`kg`

- **Output unit:** `m/s^2`
- **File:** `tension_hanging_at_rest__g.onnx`
- **Applies when:** hanging

```
inputs {'T': 2.0, 'm': 2.1}
-> 0.952381 m/s^2   (ran on tension_hanging_at_rest__g.onnx; matches the Python engine)
```

Tension in a rope holding a hanging body — solve for mass of the hanging body — `tension_hanging_at_rest.m`

$$T = mg$$

input	meaning	unit
`T`	tension in the rope	`N`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `kg`
- **File:** `tension_hanging_at_rest__m.onnx`
- **Applies when:** hanging

```
inputs {'T': 2.0, 'g': 2.1}
-> 0.952381 kg   (ran on tension_hanging_at_rest__m.onnx; matches the Python engine)
```

Thin lens equation — solve for image distance — `thin_lens.d_i`

$$\frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

input	meaning	unit
`d_o`	object distance	`m`
`f`	focal length	`m`

- **Output unit:** `m`
- **File:** `thin_lens__d_i.onnx`
- **Applies when:** lens

```
inputs {'d_o': 2.0, 'f': 2.1}
-> -42 m   (ran on thin_lens__d_i.onnx; matches the Python engine)
```

Thin lens equation — solve for object distance — `thin_lens.d_o`

$$\frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

input	meaning	unit
`d_i`	image distance	`m`
`f`	focal length	`m`

- **Output unit:** `m`
- **File:** `thin_lens__d_o.onnx`
- **Applies when:** lens

```
inputs {'d_i': 2.0, 'f': 2.1}
-> -42 m   (ran on thin_lens__d_o.onnx; matches the Python engine)
```

Thin lens equation — solve for focal length — `thin_lens.f`

$$\frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

input	meaning	unit
`d_o`	object distance	`m`
`d_i`	image distance	`m`

- **Output unit:** `m`
- **File:** `thin_lens__f.onnx`
- **Applies when:** lens

```
inputs {'d_o': 2.0, 'd_i': 2.1}
-> 1.02439 m   (ran on thin_lens__f.onnx; matches the Python engine)
```

Time to the top of a vertical throw — solve for gravitational acceleration — `time_to_top_vertical_throw.g`

$$t = \frac{v_0}{g}$$

input	meaning	unit
`t`	time from launch to the highest point	`s`
`v0`	launch speed	`m/s`

- **Output unit:** `m/s^2`
- **File:** `time_to_top_vertical_throw__g.onnx`
- **Applies when:** thrown_up

```
inputs {'t': 2.0, 'v0': 2.1}
-> 1.05 m/s^2   (ran on time_to_top_vertical_throw__g.onnx; matches the Python engine)
```

Time to the top of a vertical throw — solve for time from launch to the highest point — `time_to_top_vertical_throw.t`

$$t = \frac{v_0}{g}$$

input	meaning	unit
`v0`	launch speed	`m/s`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `s`
- **File:** `time_to_top_vertical_throw__t.onnx`
- **Applies when:** thrown_up

```
inputs {'v0': 2.0, 'g': 2.1}
-> 0.952381 s   (ran on time_to_top_vertical_throw__t.onnx; matches the Python engine)
```

Time to the top of a vertical throw — solve for launch speed — `time_to_top_vertical_throw.v0`

$$t = \frac{v_0}{g}$$

input	meaning	unit
`t`	time from launch to the highest point	`s`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `m/s`
- **File:** `time_to_top_vertical_throw__v0.onnx`
- **Applies when:** thrown_up

```
inputs {'t': 2.0, 'g': 2.1}
-> 4.2 m/s   (ran on time_to_top_vertical_throw__v0.onnx; matches the Python engine)
```

Torque of a force — solve for applied force — `torque_lever.F`

$$\tau = rF\sin(\theta)$$

input	meaning	unit
`tau`	torque about the pivot	`N*m`
`r`	distance from the pivot to the point of application	`m`
`theta`	angle between r and F	`rad`

- **Output unit:** `N`
- **File:** `torque_lever__F.onnx`
- **Applies when:** pivot

```
inputs {'tau': 2.0, 'r': 2.1, 'theta': 2.2}
-> 1.17797 N   (ran on torque_lever__F.onnx; matches the Python engine)
```

Torque of a force — solve for distance from the pivot to the point of application — `torque_lever.r`

$$\tau = rF\sin(\theta)$$

input	meaning	unit
`tau`	torque about the pivot	`N*m`
`F`	applied force	`N`
`theta`	angle between r and F	`rad`

- **Output unit:** `m`
- **File:** `torque_lever__r.onnx`
- **Applies when:** pivot

```
inputs {'tau': 2.0, 'F': 2.1, 'theta': 2.2}
-> 1.17797 m   (ran on torque_lever__r.onnx; matches the Python engine)
```

Torque of a force — solve for torque about the pivot — `torque_lever.tau`

$$\tau = rF\sin(\theta)$$

input	meaning	unit
`r`	distance from the pivot to the point of application	`m`
`F`	applied force	`N`
`theta`	angle between r and F	`rad`

- **Output unit:** `N*m`
- **File:** `torque_lever__tau.onnx`
- **Applies when:** pivot

```
inputs {'r': 2.0, 'F': 2.1, 'theta': 2.2}
-> 3.39568 N*m   (ran on torque_lever__tau.onnx; matches the Python engine)
```

Torque of a force — solve for angle between r and F — `torque_lever.theta`

$$\tau = rF\sin(\theta)$$

input	meaning	unit
`tau`	torque about the pivot	`N*m`
`r`	distance from the pivot to the point of application	`m`
`F`	applied force	`N`

- **Output unit:** `rad`
- **File:** `torque_lever__theta.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** pivot

Potential difference in a uniform field — solve for uniform field strength — `uniform_field_potential.Ef`

$$V = E_f d$$

input	meaning	unit
`V`	potential difference between the plates	`V`
`d`	separation along the field	`m`

- **Output unit:** `N/C`
- **File:** `uniform_field_potential__Ef.onnx`
- **Applies when:** uniform_field

```
inputs {'V': 2.0, 'd': 2.1}
-> 0.952381 N/C   (ran on uniform_field_potential__Ef.onnx; matches the Python engine)
```

Potential difference in a uniform field — solve for potential difference between the plates — `uniform_field_potential.V`

$$V = E_f d$$

input	meaning	unit
`Ef`	uniform field strength	`N/C`
`d`	separation along the field	`m`

- **Output unit:** `V`
- **File:** `uniform_field_potential__V.onnx`
- **Applies when:** uniform_field

```
inputs {'Ef': 2.0, 'd': 2.1}
-> 4.2 V   (ran on uniform_field_potential__V.onnx; matches the Python engine)
```

Potential difference in a uniform field — solve for separation along the field — `uniform_field_potential.d`

$$V = E_f d$$

input	meaning	unit
`V`	potential difference between the plates	`V`
`Ef`	uniform field strength	`N/C`

- **Output unit:** `m`
- **File:** `uniform_field_potential__d.onnx`
- **Applies when:** uniform_field

```
inputs {'V': 2.0, 'Ef': 2.1}
-> 0.952381 m   (ran on uniform_field_potential__d.onnx; matches the Python engine)
```

Motion at constant velocity — solve for distance travelled — `uniform_motion.d`

$$d = vt$$

input	meaning	unit
`v`	constant speed	`m/s`
`t`	elapsed time	`s`

- **Output unit:** `m`
- **File:** `uniform_motion__d.onnx`

```
inputs {'v': 2.0, 't': 2.1}
-> 4.2 m   (ran on uniform_motion__d.onnx; matches the Python engine)
```

Motion at constant velocity — solve for elapsed time — `uniform_motion.t`

$$d = vt$$

input	meaning	unit
`d`	distance travelled	`m`
`v`	constant speed	`m/s`

- **Output unit:** `s`
- **File:** `uniform_motion__t.onnx`

```
inputs {'d': 2.0, 'v': 2.1}
-> 0.952381 s   (ran on uniform_motion__t.onnx; matches the Python engine)
```

Motion at constant velocity — solve for constant speed — `uniform_motion.v`

$$d = vt$$

input	meaning	unit
`d`	distance travelled	`m`
`t`	elapsed time	`s`

- **Output unit:** `m/s`
- **File:** `uniform_motion__v.onnx`

```
inputs {'d': 2.0, 't': 2.1}
-> 0.952381 m/s   (ran on uniform_motion__v.onnx; matches the Python engine)
```

Speed after a displacement under constant acceleration — solve for acceleration — `velocity_displacement.a`

$$v^2 = v_0^2 + 2ax$$

input	meaning	unit
`v`	final speed	`m/s`
`v0`	initial speed	`m/s`
`x`	displacement	`m`

- **Output unit:** `m/s^2`
- **File:** `velocity_displacement__a.onnx`

```
inputs {'v': 2.0, 'v0': 2.1, 'x': 2.2}
-> -0.0931818 m/s^2   (ran on velocity_displacement__a.onnx; matches the Python engine)
```

Speed after a displacement under constant acceleration — solve for final speed — `velocity_displacement.v`

$$v^2 = v_0^2 + 2ax$$

input	meaning	unit
`v0`	initial speed	`m/s`
`a`	acceleration	`m/s^2`
`x`	displacement	`m`

- **Output unit:** `m/s`
- **File:** `velocity_displacement__v.onnx`
- **Branches:** 2 — take the one that is nonnegative

```
inputs {'v0': 2.0, 'a': 2.1, 'x': 2.2}
-> -3.63868, 3.63868 m/s   (ran on velocity_displacement__v.onnx; matches the Python engine)
```

Speed after a displacement under constant acceleration — solve for initial speed — `velocity_displacement.v0`

$$v^2 = v_0^2 + 2ax$$

input	meaning	unit
`v`	final speed	`m/s`
`a`	acceleration	`m/s^2`
`x`	displacement	`m`

- **Output unit:** `m/s`

- **File:** `velocity_displacement__v0.onnx`
- **Branches:** 2 — take the one that is nonnegative

Speed after a displacement under constant acceleration — solve for displacement — `velocity_displacement.x`

$$v^2 = v_0^2 + 2ax$$

input	meaning	unit
`v`	final speed	`m/s`
`v0`	initial speed	`m/s`
`a`	acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `velocity_displacement__x.onnx`

```
inputs {'v': 2.0, 'v0': 2.1, 'a': 2.2}
-> -0.0931818 m   (ran on velocity_displacement__x.onnx; matches the Python engine)
```

Velocity under constant acceleration — solve for acceleration — `velocity_time.a`

$$v = v_0 + at$$

input	meaning	unit
`v`	final velocity	`m/s`
`v0`	initial velocity	`m/s`
`t`	elapsed time	`s`

- **Output unit:** `m/s^2`
- **File:** `velocity_time__a.onnx`

```
inputs {'v': 2.0, 'v0': 2.1, 't': 2.2}
-> -0.0454545 m/s^2   (ran on velocity_time__a.onnx; matches the Python engine)
```

Velocity under constant acceleration — solve for elapsed time — `velocity_time.t`

$$v = v_0 + at$$

input	meaning	unit
`v`	final velocity	`m/s`
`v0`	initial velocity	`m/s`
`a`	acceleration	`m/s^2`

- **Output unit:** `s`
- **File:** `velocity_time__t.onnx`

Velocity under constant acceleration — solve for final velocity — `velocity_time.v`

$$v = v_0 + at$$

input	meaning	unit
`v0`	initial velocity	`m/s`
`a`	acceleration	`m/s^2`
`t`	elapsed time	`s`

- **Output unit:** `m/s`
- **File:** `velocity_time__v.onnx`

```
inputs {'v0': 2.0, 'a': 2.1, 't': 2.2}
-> 6.62 m/s   (ran on velocity_time__v.onnx; matches the Python engine)
```

Velocity under constant acceleration — solve for initial velocity — `velocity_time.v0`

$$v = v_0 + at$$

input	meaning	unit
`v`	final velocity	`m/s`
`a`	acceleration	`m/s^2`
`t`	elapsed time	`s`

- **Output unit:** `m/s`
- **File:** `velocity_time__v0.onnx`

```
inputs {'v': 2.0, 'a': 2.1, 't': 2.2}
-> -2.62 m/s   (ran on velocity_time__v0.onnx; matches the Python engine)
```

Thermal expansion of volume — solve for original volume — `volume_expansion.V0`

$$d_V = \beta V_0 d_T$$

input	meaning	unit
`dV`	change in volume	`m^3`
`beta`	coefficient of volume expansion	`1/K`
`dT`	temperature change	`K`

- **Output unit:** `m^3`
- **File:** `volume_expansion__V0.onnx`
- **Applies when:** thermal_expansion

```
inputs {'dV': 2.0, 'beta': 2.1, 'dT': 2.2}
-> 0.4329 m^3   (ran on volume_expansion__V0.onnx; matches the Python engine)
```

Thermal expansion of volume — solve for coefficient of volume expansion — `volume_expansion.beta`

$$d_V = \beta V_0 d_T$$

input	meaning	unit
`dV`	change in volume	`m^3`
`V0`	original volume	`m^3`
`dT`	temperature change	`K`

- **Output unit:** `1/K`
- **File:** `volume_expansion__beta.onnx`
- **Applies when:** thermal_expansion

```
inputs {'dV': 2.0, 'V0': 2.1, 'dT': 2.2}
-> 0.4329 1/K   (ran on volume_expansion__beta.onnx; matches the Python engine)
```

Thermal expansion of volume — solve for temperature change — `volume_expansion.dT`

$$d_V = \beta V_0 d_T$$

input	meaning	unit
`dV`	change in volume	`m^3`
`beta`	coefficient of volume expansion	`1/K`
`V0`	original volume	`m^3`

- **Output unit:** `K`
- **File:** `volume_expansion__dT.onnx`
- **Applies when:** thermal_expansion

```
inputs {'dV': 2.0, 'beta': 2.1, 'V0': 2.2}
-> 0.4329 K   (ran on volume_expansion__dT.onnx; matches the Python engine)
```

Thermal expansion of volume — solve for change in volume — `volume_expansion.dV`

$$d_V = \beta V_0 d_T$$

input	meaning	unit
`beta`	coefficient of volume expansion	`1/K`
`V0`	original volume	`m^3`
`dT`	temperature change	`K`

- **Output unit:** `m^3`
- **File:** `volume_expansion__dV.onnx`
- **Applies when:** thermal_expansion

```
inputs {'beta': 2.0, 'V0': 2.1, 'dT': 2.2}
-> 9.24 m^3   (ran on volume_expansion__dV.onnx; matches the Python engine)
```

Speed of a periodic wave — solve for frequency — `wave_speed.f`

$$v = f\lambda$$

input	meaning	unit
`v`	wave speed	`m/s`
`lam`	wavelength	`m`

- **Output unit:** `Hz`
- **File:** `wave_speed__f.onnx`

```
inputs {'v': 2.0, 'lam': 2.1}
-> 0.952381 Hz   (ran on wave_speed__f.onnx; matches the Python engine)
```

Speed of a periodic wave — solve for wavelength — `wave_speed.lam`

$$v = f\lambda$$

input	meaning	unit
`v`	wave speed	`m/s`
`f`	frequency	`Hz`

- **Output unit:** `m`
- **File:** `wave_speed__lam.onnx`

```
inputs {'v': 2.0, 'f': 2.1}
-> 0.952381 m   (ran on wave_speed__lam.onnx; matches the Python engine)
```

Speed of a periodic wave — solve for wave speed — `wave_speed.v`

$$v = f\lambda$$

input	meaning	unit
`f`	frequency	`Hz`
`lam`	wavelength	`m`

- **Output unit:** `m/s`
- **File:** `wave_speed__v.onnx`

```
inputs {'f': 2.0, 'lam': 2.1}
-> 4.2 m/s   (ran on wave_speed__v.onnx; matches the Python engine)
```

Wavelength of light in a medium — solve for wavelength in vacuum — `wavelength_in_medium.lam`

$$\lambda_n = \frac{\lambda}{n}$$

input	meaning	unit
`lam_n`	wavelength in the medium	`m`
`n`	index of refraction	`1`

- **Output unit:** `m`
- **File:** `wavelength_in_medium__lam.onnx`
- **Applies when:** medium

```
inputs {'lam_n': 2.0, 'n': 2.1}
-> 4.2 m   (ran on wavelength_in_medium__lam.onnx; matches the Python engine)
```

Wavelength of light in a medium — solve for wavelength in the medium — `wavelength_in_medium.lam_n`

$$\lambda_n = \frac{\lambda}{n}$$

input	meaning	unit
`lam`	wavelength in vacuum	`m`
`n`	index of refraction	`1`

- **Output unit:** `m`
- **File:** `wavelength_in_medium__lam_n.onnx`
- **Applies when:** medium

```
inputs {'lam': 2.0, 'n': 2.1}
-> 0.952381 m   (ran on wavelength_in_medium__lam_n.onnx; matches the Python engine)
```

Wavelength of light in a medium — solve for index of refraction — `wavelength_in_medium.n`

$$\lambda_n = \frac{\lambda}{n}$$

input	meaning	unit
`lam_n`	wavelength in the medium	`m`
`lam`	wavelength in vacuum	`m`

- **Output unit:** `1`
- **File:** `wavelength_in_medium__n.onnx`
- **Applies when:** medium

```
inputs {'lam_n': 2.0, 'lam': 2.1}
-> 1.05 1   (ran on wavelength_in_medium__n.onnx; matches the Python engine)
```

Weight in a uniform gravitational field — solve for weight — `weight.W`

$$W = mg$$

input	meaning	unit
`m`	mass	`kg`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `N`
- **File:** `weight__W.onnx`

```
inputs {'m': 2.0, 'g': 2.1}
-> 4.2 N   (ran on weight__W.onnx; matches the Python engine)
```

Weight in a uniform gravitational field — solve for gravitational acceleration — `weight.g`

$$W = mg$$

input	meaning	unit
`W`	weight	`N`
`m`	mass	`kg`

- **Output unit:** `m/s^2`
- **File:** `weight__g.onnx`

```
inputs {'W': 2.0, 'm': 2.1}
-> 0.952381 m/s^2   (ran on weight__g.onnx; matches the Python engine)
```

Weight in a uniform gravitational field — solve for mass — `weight.m`

$$W = mg$$

input	meaning	unit
`W`	weight	`N`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `kg`
- **File:** `weight__m.onnx`

```
inputs {'W': 2.0, 'g': 2.1}
-> 0.952381 kg   (ran on weight__m.onnx; matches the Python engine)
```

Component of the weight along an incline — solve for component of the weight along the incline — `weight_component_along_incline.F`

$$F = mg\sin(\theta)$$

input	meaning	unit
`m`	mass	`kg`
`g`	gravitational acceleration	`m/s^2`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `N`
- **File:** `weight_component_along_incline__F.onnx`
- **Applies when:** inclined

```
inputs {'m': 2.0, 'g': 2.1, 'theta': 2.2}
-> 3.39568 N   (ran on weight_component_along_incline__F.onnx; matches the Python engine)
```

Component of the weight along an incline — solve for gravitational acceleration — `weight_component_along_incline.g`

$$F = mg\sin(\theta)$$

input	meaning	unit
`F`	component of the weight along the incline	`N`
`m`	mass	`kg`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `m/s^2`
- **File:** `weight_component_along_incline__g.onnx`
- **Applies when:** inclined

```
inputs {'F': 2.0, 'm': 2.1, 'theta': 2.2}
-> 1.17797 m/s^2   (ran on weight_component_along_incline__g.onnx; matches the Python engine)
```

Component of the weight along an incline — solve for mass — `weight_component_along_incline.m`

$$F = mg\sin(\theta)$$

input	meaning	unit
`F`	component of the weight along the incline	`N`

input	meaning	unit
`g`	gravitational acceleration	`m/s^2`
`theta`	angle of the incline to the horizontal	`rad`

- **Output unit:** `kg`
- **File:** `weight_component_along_incline__m.onnx`
- **Applies when:** inclined

```
inputs {'F': 2.0, 'g': 2.1, 'theta': 2.2}
-> 1.17797 kg   (ran on weight_component_along_incline__m.onnx; matches the Python engine)
```

Component of the weight along an incline — solve for angle of the incline to the horizontal — `weight_component_along_incline.theta`

$$F = mg\sin\left(\theta\right)$$

input	meaning	unit
`F`	component of the weight along the incline	`N`
`m`	mass	`kg`
`g`	gravitational acceleration	`m/s^2`

- **Output unit:** `rad`
- **File:** `weight_component_along_incline__theta.onnx`
- **Branches:** 2 — take the one that is nonnegative
- **Applies when:** inclined

Work by a constant force — solve for force — `work.F`

$$W = Fd\cos\left(\theta\right)$$

input	meaning	unit
`W`	work done	`J`
`d`	displacement	`m`
`theta`	angle between force and displacement	`rad`

- **Output unit:** `N`
- **File:** `work__F.onnx`

```
inputs {'W': 2.0, 'd': 2.1, 'theta': 2.2}
-> -1.61832 N   (ran on work__F.onnx; matches the Python engine)
```

Work by a constant force — solve for work done — `work.W`

$$W = Fd\cos\left(\theta\right)$$

input	meaning	unit
`F`	force	`N`
`d`	displacement	`m`
`theta`	angle between force and displacement	`rad`

- **Output unit:** `J`
- **File:** `work__W.onnx`

```
inputs {'F': 2.0, 'd': 2.1, 'theta': 2.2}
-> -2.4717 J   (ran on work__W.onnx; matches the Python engine)
```

Work by a constant force — solve for displacement — `work.d`

$$W = Fd\cos(\theta)$$

input	meaning	unit
`W`	work done	`J`
`F`	force	`N`
`theta`	angle between force and displacement	`rad`

- **Output unit:** `m`
- **File:** `work__d.onnx`

```
inputs {'W': 0.5, 'F': 0.6, 'theta': 0.7}
-> 1.08955 m   (ran on work__d.onnx; matches the Python engine)
```

Work by a constant force — solve for angle between force and displacement — `work.theta`

$$W = Fd\cos(\theta)$$

input	meaning	unit
`W`	work done	`J`
`F`	force	`N`
`d`	displacement	`m`

- **Output unit:** `rad`
- **File:** `work__theta.onnx`
- **Branches:** 2 — take the one that is any